Topology Optimization
Lecture 8 – Paper G1

On projection methods, convergence and robust formulations in topology optimization

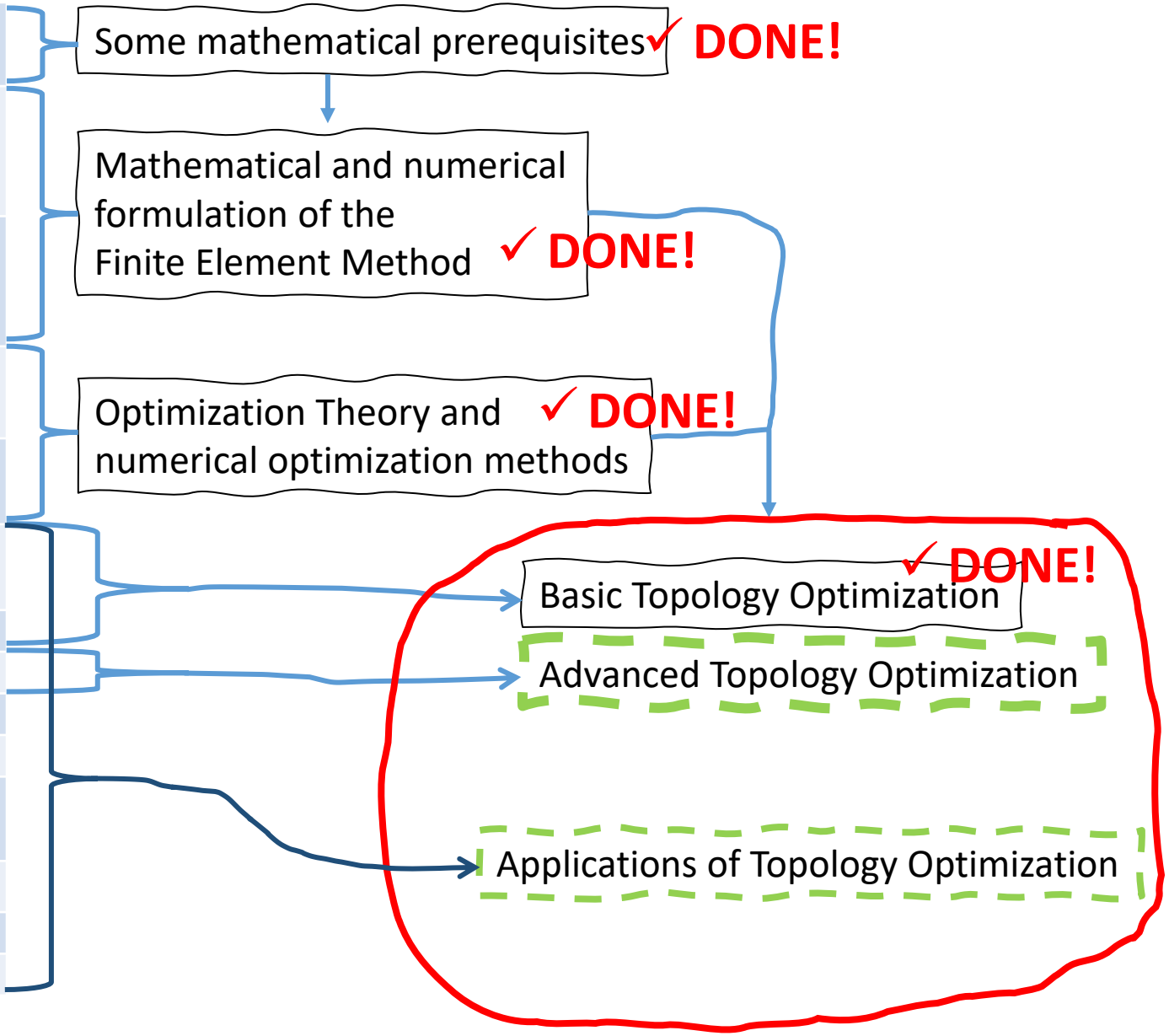
F. Wang, B. S. Lazarov & O. Sigmund

Struct. Multidisc. Optim. 43, 767–784 (2011)

https://doi.org/10.1007/s00158-010-0602-y

Course overview

No.	Topics
1	Introduction Function and vector spaces, Interpolation, L^2 projection
2	The Finite Element Method for PDEs 1: Poisson's Equation, The Finite Element Method, Properties of the Finite Element Method
3	The Finite Element Method for PDEs 2: Index notation, Linear Elasticity, Non-linear Poisson's equation, Non-linear Elasticity
4	Constrained Optimization 1: Optimality, KKT Theorem, Numerical Optimization Algorithms
5	Constrained Optimization 2: Method of Moving Asymptotes, MPI and PETSc
6	Introduction to Topology Optimization: The idea, Filtering and Projection, Python implementation
7	Gradients
8	Robust topology optimization, More on MPI
9	Wave equations
10	Acoustic waves and more on Robust Optimization
11	Penalty and Augmented Lagrangian Methods Stress-constraints in Topology Optimization 1
12	Stress-constraints in Topology Optimization 2
13	Light concentration using nanoparticle design
14	Heat conduction



The main points of the paper

- The paper discusses filtering and projection techniques.
- It is argued that the simple filtering and projection, as we have used so far, does not always lead to well-defined feature sizes in the optimized design.
- The paper presents a “robust” scheme, which leads to well-defined length scales for both solid and voids.
- The new scheme is applied to 3 test problems:
  - Compliant force inverter.
  - Heat conduction.
  - Compliant gripper.
- It is shown how the new scheme can solve problems which has previously resulted in problematic designs, e.g., 1-node hinges in compliant mechanisms.
- The “penalty” to pay is that 3 finite element problems must be solved to evaluate the cost function, compared to 1 for the “normal” scheme.

Problem definitions

- All 3 problems (force inverter, heat conduction & gripper) are formulated as

$$
\begin{aligned}
\min_{\rho} &: f(\rho) = \mathbf{l}^{\mathsf{T}}\mathbf{u} \\
s.t. &: \mathbf{K}\mathbf{u} = \mathbf{f} \\
&: f_v(\rho) = \frac{\sum_{i\in\mathbb{N}_e} \bar{\tilde{\rho}}_i v_i}{V} \leq V^* \\
&: \mathbf{0} \leq \rho_i \leq \mathbf{1} \forall i \in \mathbb{N}_e
\end{aligned} \tag{1}
$$

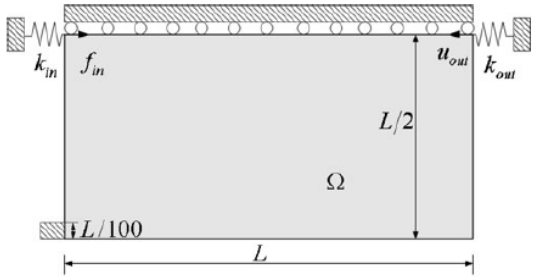
- The problems are discussed in their discrete forms, where
  - **l** is a vector used to define the cost function (next slide).
  - **u** is the state variable (displacement or temperature).
  - **K** is the stiffness matrix.
  - ρ is the design density, $\tilde{\rho}$ is the filtered density and $\bar{\tilde{\rho}}$ is the filtered and projected density.
  - V is the total volume and V^* is the allowed volume fraction.
  - v_i are the element volumes.
- A modified SIMP model with *p*=3 is used: $E_i = E_{\min} + \bar{\tilde{\rho}}_i^p (E_0 - E_{\min})$ (2)

Problem definitions

- All 3 problems (force inverter, heat conduction & gripper) are formulated as

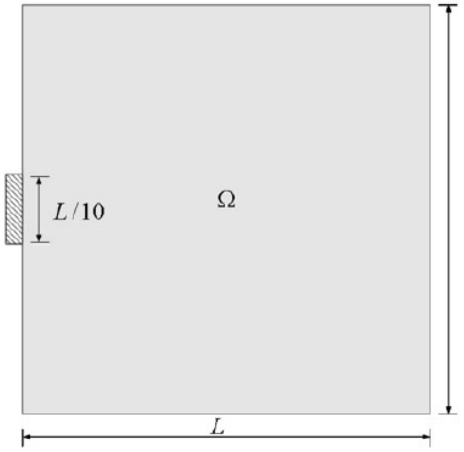
$$
\begin{aligned}
\min_{\rho} : \; & f(\rho) = \mathbf{l}^{\mathsf{T}}\mathbf{u} \\
s.t. : \; & \mathbf{K}\mathbf{u} = \mathbf{f} \\
& : f_v(\rho) = \frac{\sum_{i \in \mathbb{N}_e} \bar{\bar{\rho}}_i v_i}{V} \leq V^* \\
& : \mathbf{0} \leq \rho_i \leq \mathbf{1} \forall i \in \mathbb{N}_e
\end{aligned}
\tag{1}
$$

- The vector **l** in the cost function *f* takes different values for the 3 test problems.



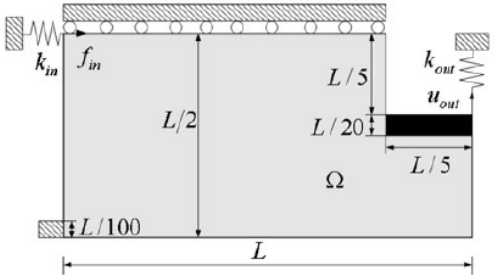
(a)

Compliant force inverter: l_i=0 for all nodes except at u_{out}, where l_i=1.



(b)

Heat conduction: **l**=**f** (the load vector, discussed later in the course…)



(c)

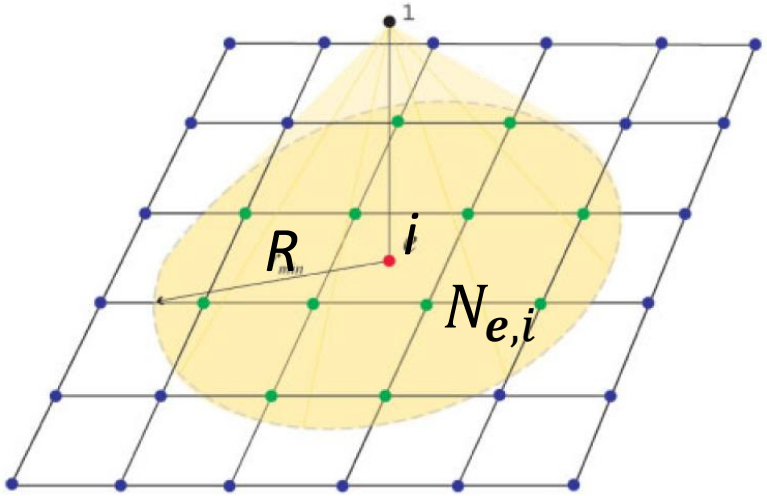
Compliant gripper: l_i=0 for all nodes except at u_{out}, where l_i=1.

Filtering

- The filtered density is calculated by a convolution filter (weighted average)

$$\tilde{\rho}_i = \frac{\sum_{j \in \mathbb{N}_{e,i}} w(\mathbf{x}_j) v_j \rho_j}{\sum_{j \in \mathbb{N}_{e,i}} w(\mathbf{x}_j) v_j} \tag{4}$$

$$w(\mathbf{x}_j) = R - |\mathbf{x}_j - \mathbf{x}_i| \tag{5}$$



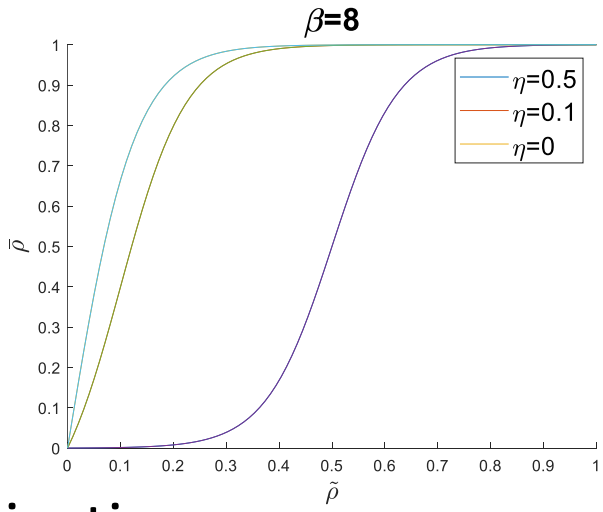
$\boldsymbol{x}_i$ are the center coordinates of cell *i*.
$N_{e,i}$ is the neighborhood of element *i* (yellow disc).

- This filter is commonly used in 2D and named a convolution filter.
- The filter is said to have “compact support”, as only elements in the neighborhood is considered to calculate the filtered density of element *i*.
- The Helmholtz filter does not have “compact support” as all elements may impact the filtered density in element *i*, even those far away but of course only by a very small amount.
- The Helmholtz filter is considered later in the paper.

Projection

- The projection used is given by

$$\bar{\tilde{\rho}}_i = \begin{cases} \eta\left[e^{-\beta(1-\tilde{\rho}_i/\eta)} - (1-\tilde{\rho}_i/\eta)e^{-\beta}\right] & 0 \le \tilde{\rho}_i \le \eta \\ (1-\eta)\left[1 - e^{-\beta(\tilde{\rho}_i-\eta)/(1-\eta)} \right. & \\ \qquad \left. + (\tilde{\rho}_i-\eta)/(1-\eta)e^{-\beta}\right] + \eta & \eta < \tilde{\rho}_i \le 1 \end{cases}$$

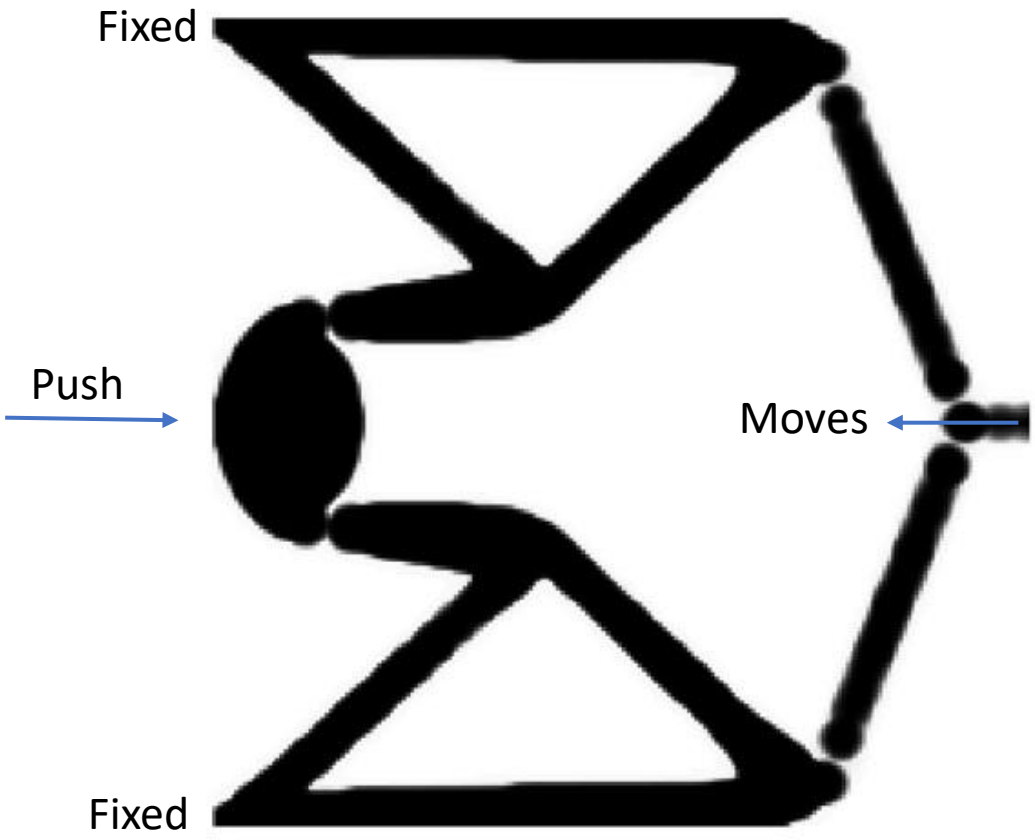


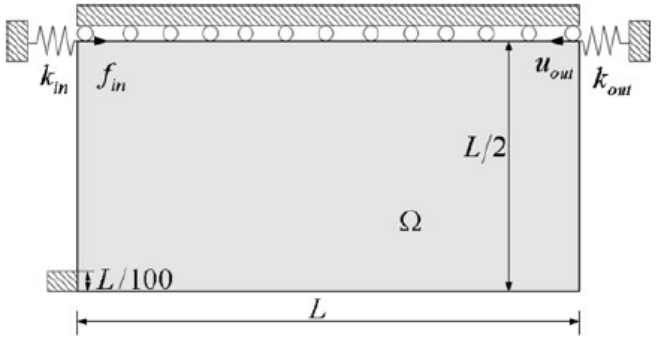
- Alternatively, they suggest to use the smoothed Heaviside projection:

$$\bar{\tilde{\rho}}_i = \frac{\tanh(\beta\eta) + \tanh(\beta(\tilde{\rho}_i - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1-\eta))} \qquad (9)$$

- When combined with the filter with compact support, both projection schemes result in
  - A solid length scale equal to the filter radius for $\eta = 0$.
  - A void length scale equal to the filter radius for $\eta = 1$.
- This is not exactly true for the Helmholtz smoothing filter, where we saw previously that $\eta = 0$ and $\eta = 1$ generally leads to very soft regions and $\eta = 0$ and $\eta = 1$ should therefore be avoided when using the Helmholtz filter.
- The scheme proposed in the paper does not (usually) use these two values and the new scheme can also be used directly with the Helmholtz filter.

Compliant force inverter

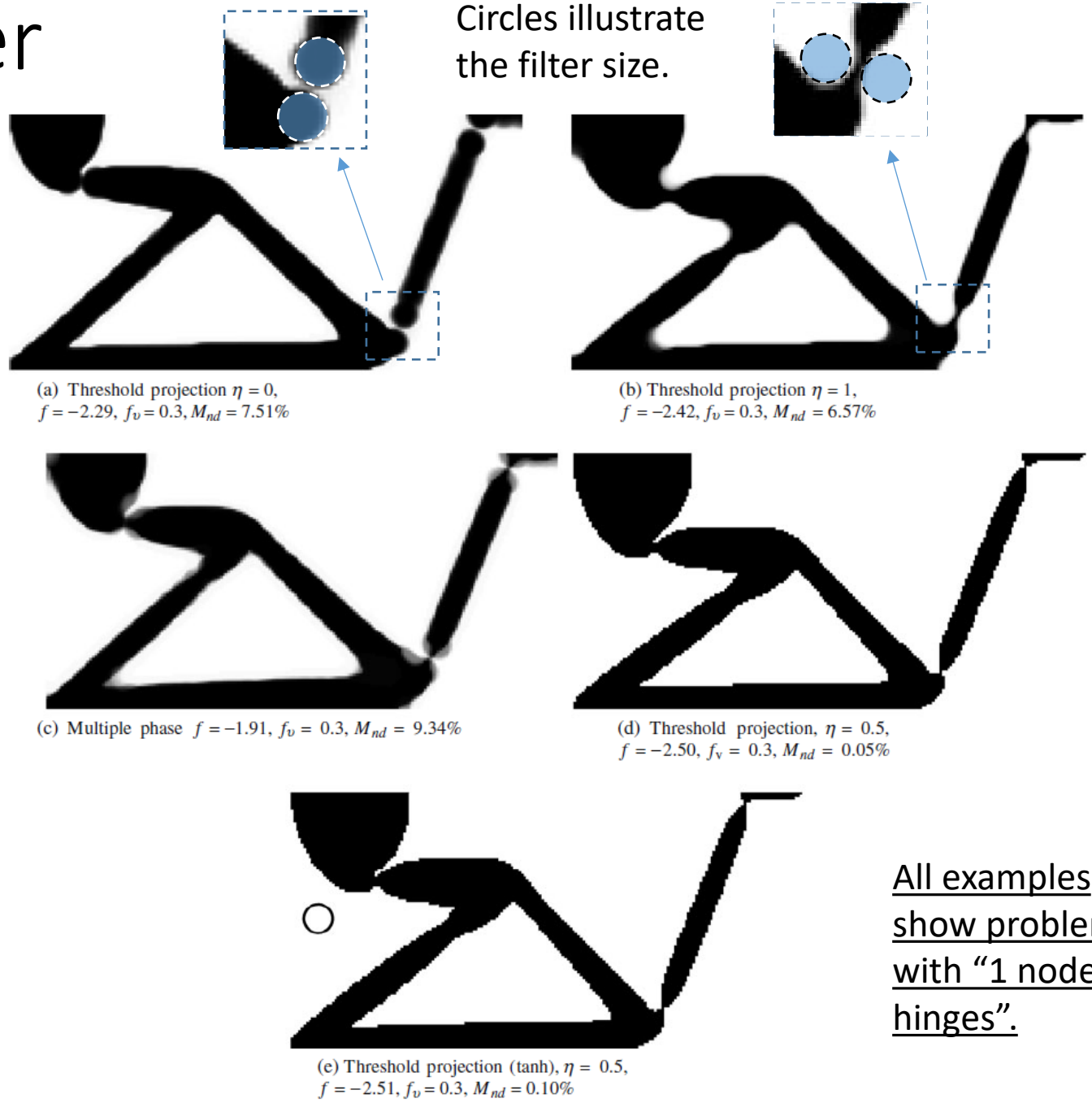




(a)

Due to symmetry, only half of the geometry is needed and shown in the results.

Circles illustrate the filter size.



(a) Threshold projection $\eta = 0$, $f = -2.29$, $f_v = 0.3$, $M_{nd} = 7.51\%$

(b) Threshold projection $\eta = 1$, $f = -2.42$, $f_v = 0.3$, $M_{nd} = 6.57\%$

(c) Multiple phase $f = -1.91$, $f_v = 0.3$, $M_{nd} = 9.34\%$

(d) Threshold projection, $\eta = 0.5$, $f = -2.50$, $f_v = 0.3$, $M_{nd} = 0.05\%$

(e) Threshold projection (tanh), $\eta = 0.5$, $f = -2.51$, $f_v = 0.3$, $M_{nd} = 0.10\%$

Fig. 3 Topology optimization of compliant inverter based on different filters. The filter support is shown with *circle* in subfigure e

All examples show problems with “1 node hinges”.

The new scheme: Robust Topology Optimization

- The problem is reformulated as

$$
\begin{aligned}
\min_{\rho} : \max \quad & \left(\mathbf{f}(\bar{\tilde{\rho}}^{e}(\rho)), \mathbf{f}(\bar{\tilde{\rho}}^{i}(\rho)), \mathbf{f}(\bar{\tilde{\rho}}^{d}(\rho))\right) \\
s.t. : \; & \mathbf{K}(\bar{\tilde{\rho}}^{e})\mathbf{u}^{e} = \mathbf{f} \\
: \; & \mathbf{K}(\bar{\tilde{\rho}}^{i})\mathbf{u}^{i} = \mathbf{f} \\
: \; & \mathbf{K}(\bar{\tilde{\rho}}^{d})\mathbf{u}^{d} = \mathbf{f} \\
: \; & f_v(\rho) = \frac{\sum_i \bar{\tilde{\rho}}_i^{d} v_i}{V} \leq V_d^{*} \\
: \; & 0 \leq \rho \leq 1
\end{aligned}
\tag{14}
$$

The "cost functions" (should be *f*, not **f**).

The load vector, **f**.

The two **f** are not the same…

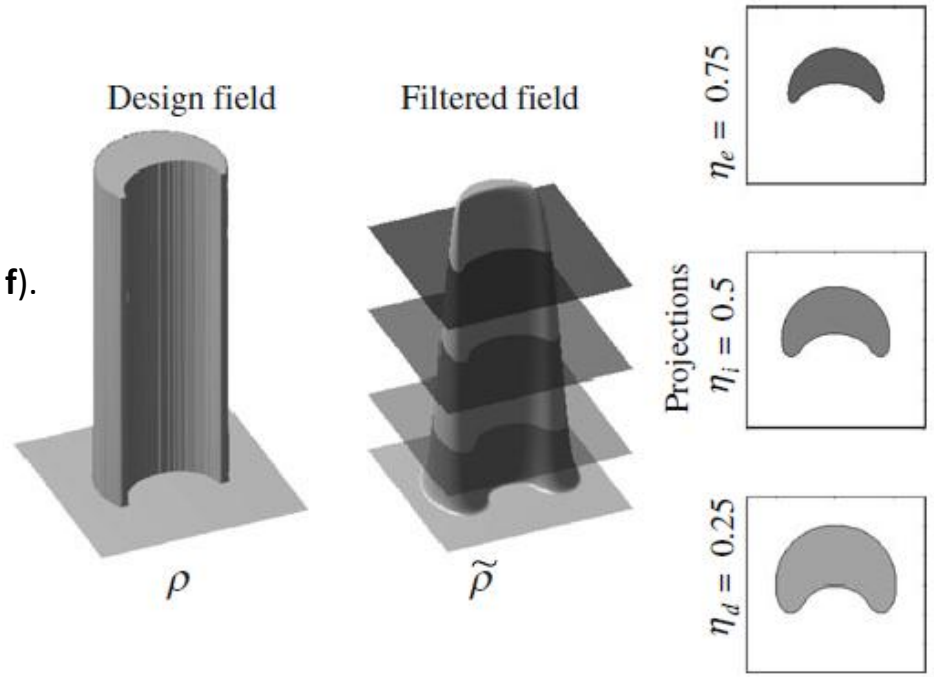


Fig. 6 Illustration of the threshold projections for different choices of the threshold value η

- $\bar{\tilde{\rho}}^{e}$ is named the eroded density.
- $\bar{\tilde{\rho}}^{i}$ is named the intermediate density, used as the "blue-print design".
- $\bar{\tilde{\rho}}^{d}$ is named the dilated density.
- η controls the 3 projection thresholds used: $\eta_d = \eta, \eta_i = 0.5\,, \eta_e = 1 - \eta$
- The problem is now a min-max problem, and 3 FEM problems must be solved to evaluate the 3 "cost functions", *f*.

The new scheme: Robust Topology Optimization

$$
\begin{aligned}
\min_{\rho} : \max \quad & \left(\mathbf{f}(\bar{\bar{\rho}}^e(\rho)), \mathbf{f}(\bar{\bar{\rho}}^i(\rho)), \mathbf{f}(\bar{\bar{\rho}}^d(\rho))\right) \\
s.t. : \; & \mathbf{K}(\bar{\bar{\rho}}^e)\mathbf{u}^e = \mathbf{f} \\
& : \mathbf{K}(\bar{\bar{\rho}}^i)\mathbf{u}^i = \mathbf{f} \\
& : \mathbf{K}(\bar{\bar{\rho}}^d)\mathbf{u}^d = \mathbf{f} \\
& : f_v(\rho) = \frac{\sum_i \bar{\bar{\rho}}_i^d v_i}{V} \leq V_d^* \\
& : 0 \leq \rho \leq 1
\end{aligned}
\tag{14}
$$

V^* is the desired volume fraction.
V_i is the current volume fraction of the intermediate design.
V_d is the current volume fraction of the dilated design.

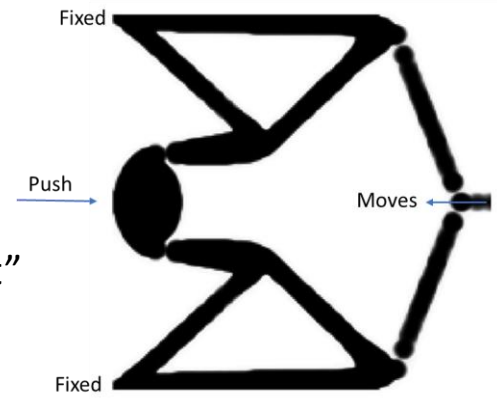
1. Initialize the design variable ρ. Set up the threshold value η and the maximum design variable change $\Delta\rho_{\max}$, the iteration counter $i = 0$, and $\beta = 1$.

2. **while** $|\Delta\rho|_\infty \leq \Delta\rho_{\max}$ and $i \leq i_{\max}$ and $\beta \leq \beta_{\max}$ **do**

   - $i = i + 1$
   - Compute $\bar{\bar{\rho}}^e$, $\bar{\bar{\rho}}^i$ and $\bar{\bar{\rho}}^d$ using (8).
   - Solve the three FEM problems $\mathbf{K}(\bar{\bar{\rho}}^i)\mathbf{u}^i = \mathbf{f}$, $\mathbf{K}(\bar{\bar{\rho}}^e)\mathbf{u}^e = \mathbf{f}$ and $\mathbf{K}(\bar{\bar{\rho}}^d)\mathbf{u}^d = \mathbf{f}$.
   - Compute the sensitivities for the three designs using (7).
   - Update the design variables.
   - **if** [mod(i,20) = 1] update the volume fraction of the dilated structure $V_d^* = \frac{V^*}{V_i} V_d$ such that the volume of the intermediate design V_i becomes equal to V^*.
   - **if** [mod(i,50) = 1 or $|\Delta\rho|_\infty \leq 0.01$] and [$\beta \leq \beta_{\max}$] **then** $\beta = 2\beta$

"However, in the present approach the intermediate design has gone through the same density filtering as the dilated and eroded designs and it is obtained as the projection for η = 0.5. The modifications eliminate numerical artifacts associated with the old approach and result in much more stable convergence of the optimization problem."

The cost function *f* in the force inverter example is the displacement in the x-direction. This should be as *negative* as possible, so "max x-displacement" of the 3 realizations is the worst performing case.



Compliant force inverter, Robust optimization

η controls the 3 projection thresholds used:

$\eta_d = \eta$

$\eta_i = 0.5$

$\eta_e = 1 - \eta$

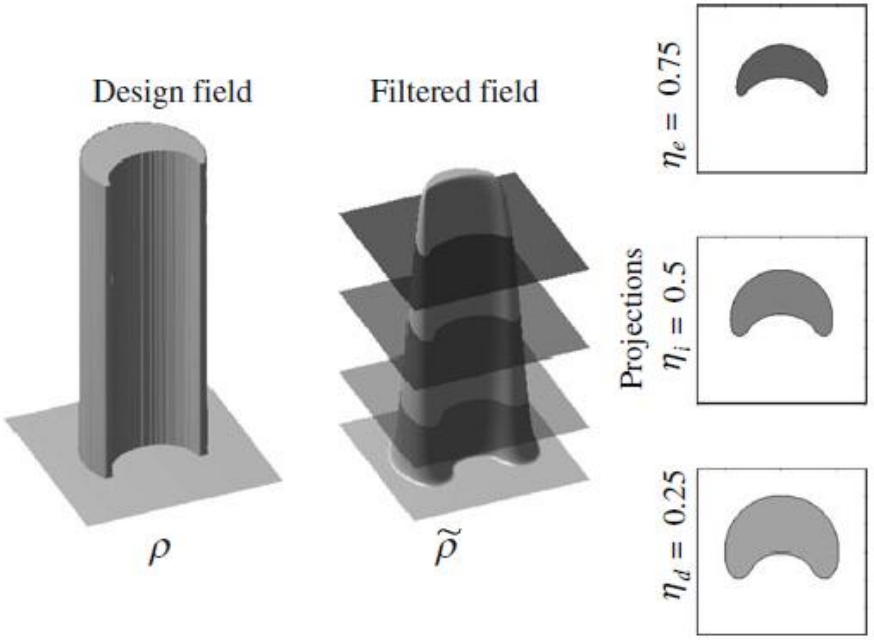


Fig. 6 Illustration of the threshold projections for different choices of the threshold value η

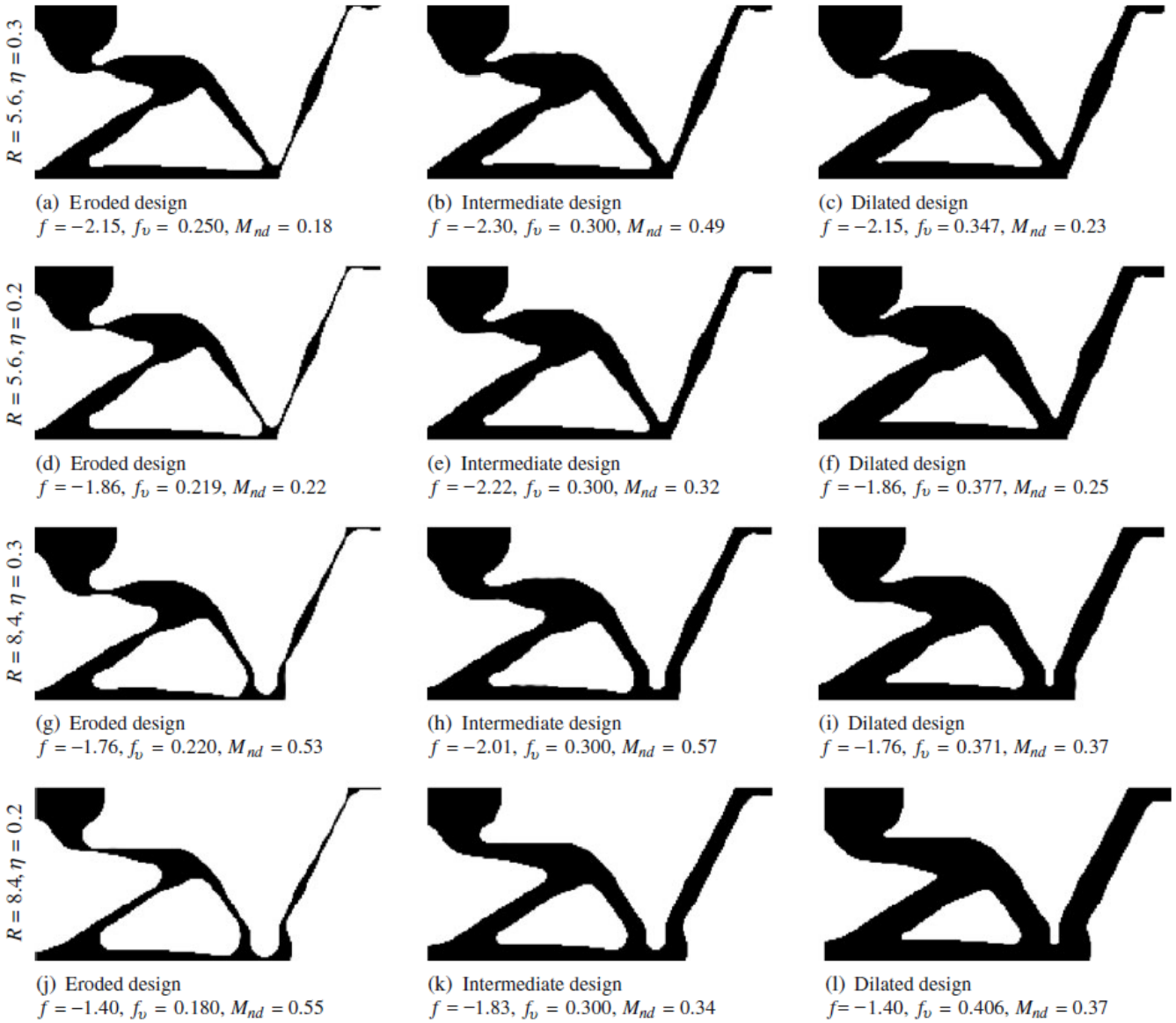


Fig. 8 Robust design of compliant inverter for different parameter sets

Minimum length scale

(read yourself if interested, not discussed in class)

$$\tilde{\rho}_i = \frac{\sum_{j \in \mathbb{N}_{e,i}} w(\mathbf{x}_j) v_j \rho_j}{\sum_{j \in \mathbb{N}_{e,i}} w(\mathbf{x}_j) v_j} \quad (4)$$

$$w(\mathbf{x}_j) = R - |\mathbf{x}_j - \mathbf{x}_i| \quad (5)$$

- For the solid region at the midpoint $x_{p,1}$ using a convolution filter:
  - If $h \geq 2R$, $\tilde{\rho}(x_{p,1}) = \rho(x_{p,1})$
  - If $h < 2R$, $\tilde{\rho}(x_{p,1}) < \rho(x_{p,1})$
- Projection with "true" Heaviside projection with threshold $\eta = \lim_{\delta \to 0}(\tilde{\rho}(x_{p,1}) - \delta) = \max(\tilde{\rho}(x)) = \tilde{\rho}_{max}$ result in a single point of density>0 (solid).
- $\tilde{\rho}_{max}$ is plotted below.
- Projection with η above the solid line, will result in void only.
- Projection with η at the solid line, will result in a single element with non-zero density.
- Projection with η below the solid line, will result in a solid region with non-zero length.
- If a 'solid point' appear in the eroded design, then $\eta_e \approx \tilde{\rho}_{max}$.
- At this value a, finite size feature will appear in η_i, with size found by looking at the interval where $\tilde{\rho} \geq \frac{1}{2}$.

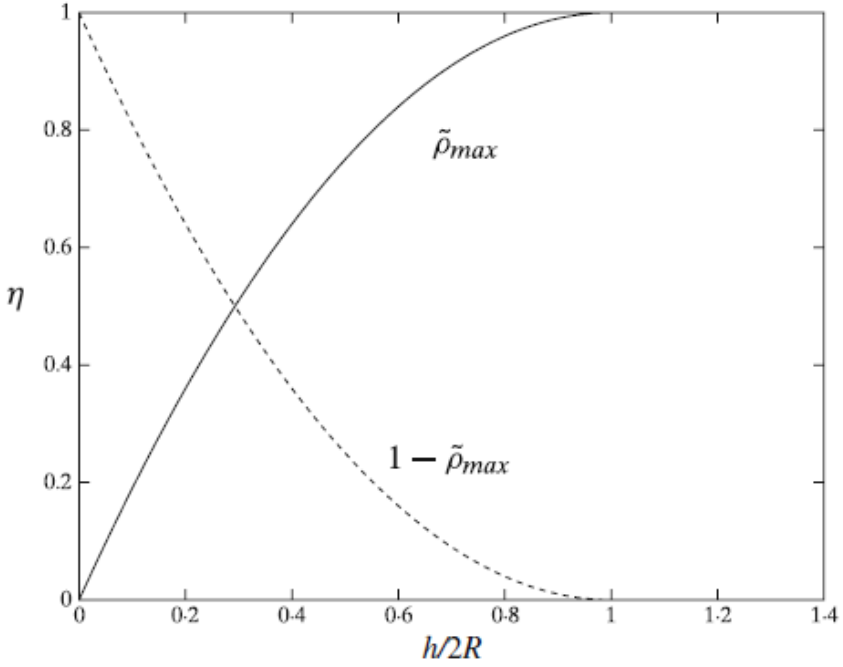


Fig. 11 Maximum/minimum of the filtered density of the solid/void phase as functions of the normalized width $h/2R$

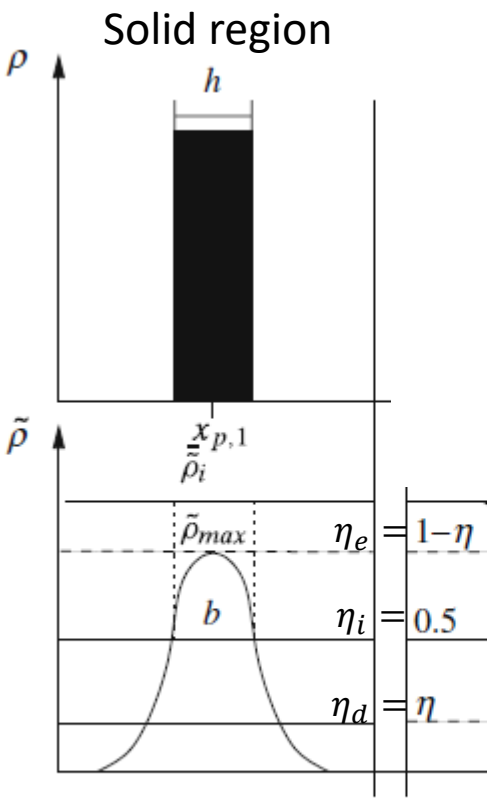


Fig. 10 Density filtering of solid and void phases and their Heaviside projections

Minimum length scale

(read yourself if interested, not discussed in class)

$$\tilde{\rho}_i = \frac{\sum_{j \in \mathbb{N}_{e,i}} w(\mathbf{x}_j) v_j \rho_j}{\sum_{j \in \mathbb{N}_{e,i}} w(\mathbf{x}_j) v_j} \quad (4)$$

$$w(\mathbf{x}_j) = R - |\mathbf{x}_j - \mathbf{x}_i| \quad (5)$$

- For the void region at the midpoint $x_{p,2}$ using a convolution filter:
  - If $h \geq 2R$, $\tilde{\rho}(x_{p,2}) = \rho(x_{p,2})$
  - If $h < 2R$, $\tilde{\rho}(x_{p,2}) > \rho(x_{p,2})$
- Projection with "true" Heaviside projection with threshold $\eta = \lim_{\delta \to 0}(\tilde{\rho}(x_{p,2}) + \delta) = \max(1 - \tilde{\rho}(x)) = 1 - \tilde{\rho}_{max}$ result in a single point of zero density (void).
- 1-$\tilde{\rho}_{max}$ is plotted below.
- Projection with η below the dashed line, will result in solid only.
- Projection with η at the dashed line, will result in a single element with zero density.
- Projection with η above the dashed line, will result in a void region with non-zero length.
- If a 'void point' appear in the dilated design, then $\eta_d \approx 1 - \tilde{\rho}_{max}$.
- At this value, a finite size void will appear in η_i, with size found by looking at the interval where $\tilde{\rho} \leq \frac{1}{2}$.

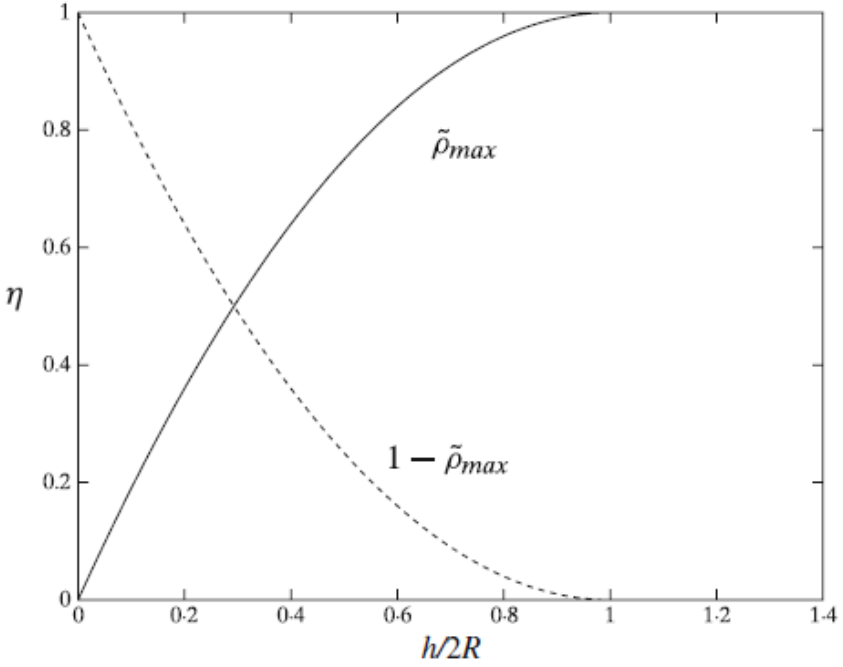


Fig. 11 Maximum/minimum of the filtered density of the solid/void phase as functions of the normalized width $h/2R$

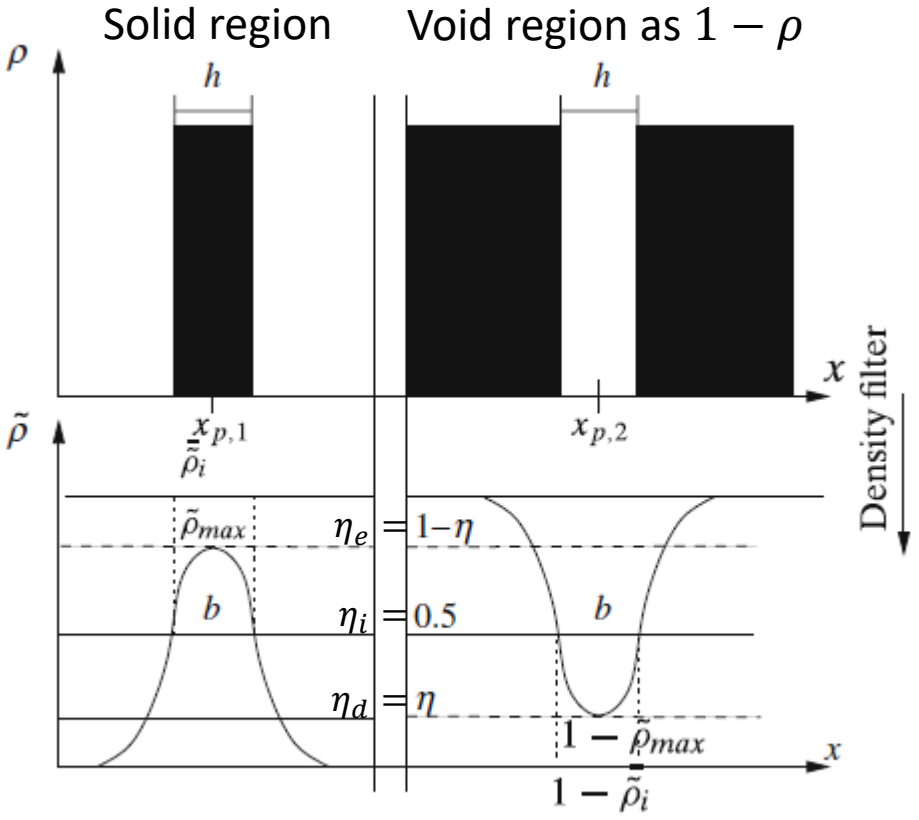


Fig. 10 Density filtering of solid and void phases and their Heaviside projections

Minimum length scale

(read yourself if interested, not discussed in class)

- Assume the overall topology of the eroded, intermediate and dilated designs are the same.
- If there is a single solid point in the eroded design, there will be a larger area of material in the intermediate design.
- If there is a void point in the dilated design, there will be a void of larger area in the intermediate design.
- η thus determines the minimum solid size and void size in the intermediate design.
- The relation between η and the feature sizes in the intermediate design can be found by evaluating the interval where $\tilde{\rho} \geq 1/2$ for solid and $\tilde{\rho} \leq 1/2$ for void.
- The bottom right plot show this relationship, which, can be used to determine the η value to get a certain feature size in the intermediate design.

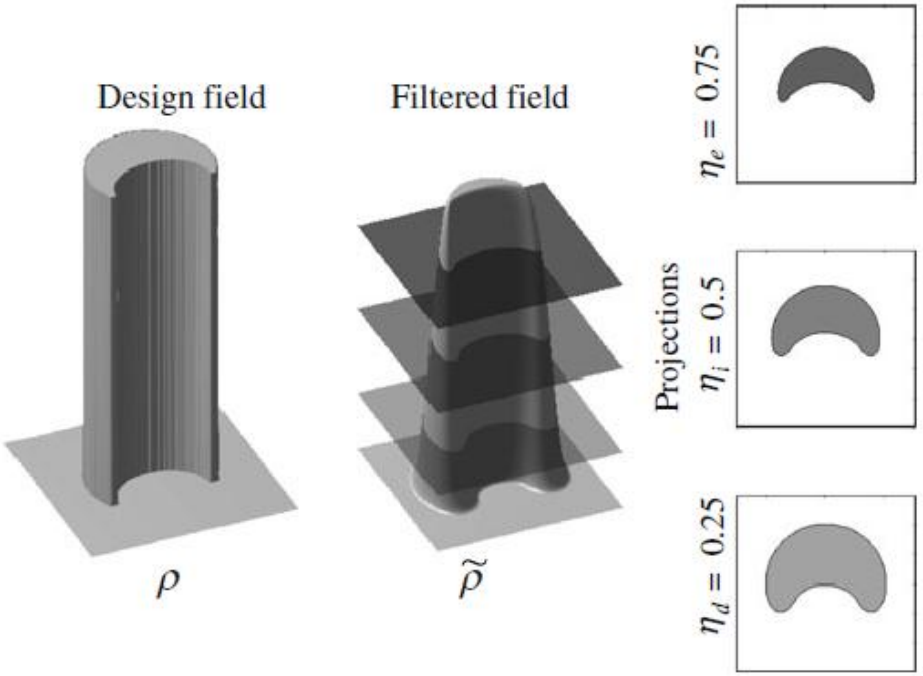


Fig. 6 Illustration of the threshold projections for different choices of the threshold value η

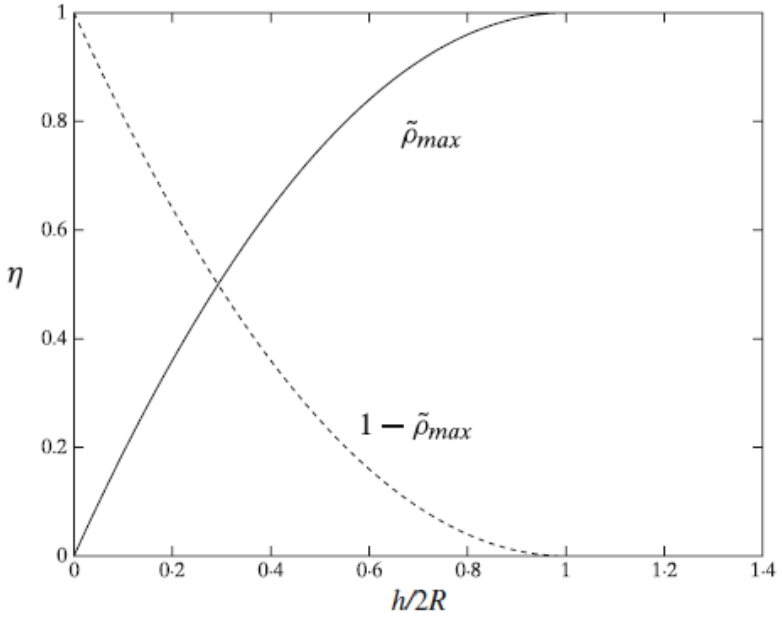
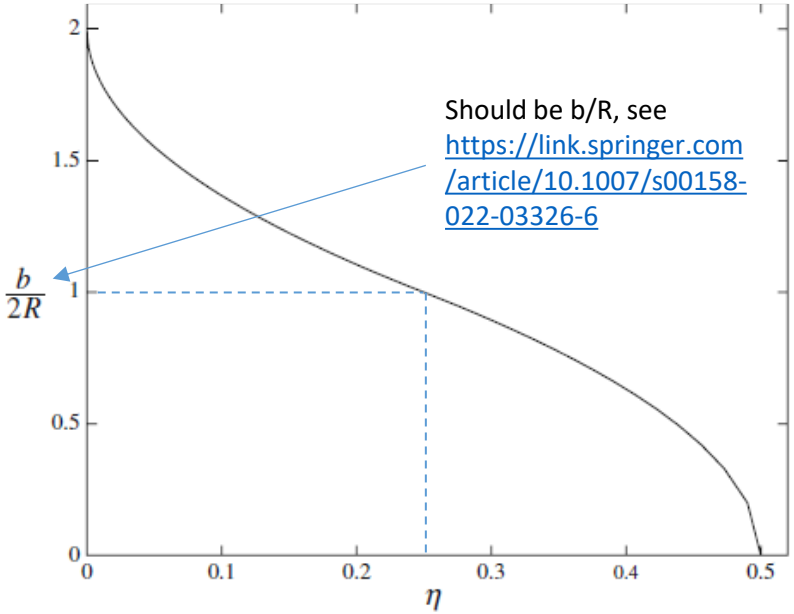


Fig. 11 Maximum/minimum of the filtered density of the solid/void phase as functions of the normalized width $h/2R$



Should be b/R, see https://link.springer.com/article/10.1007/s00158-022-03326-6

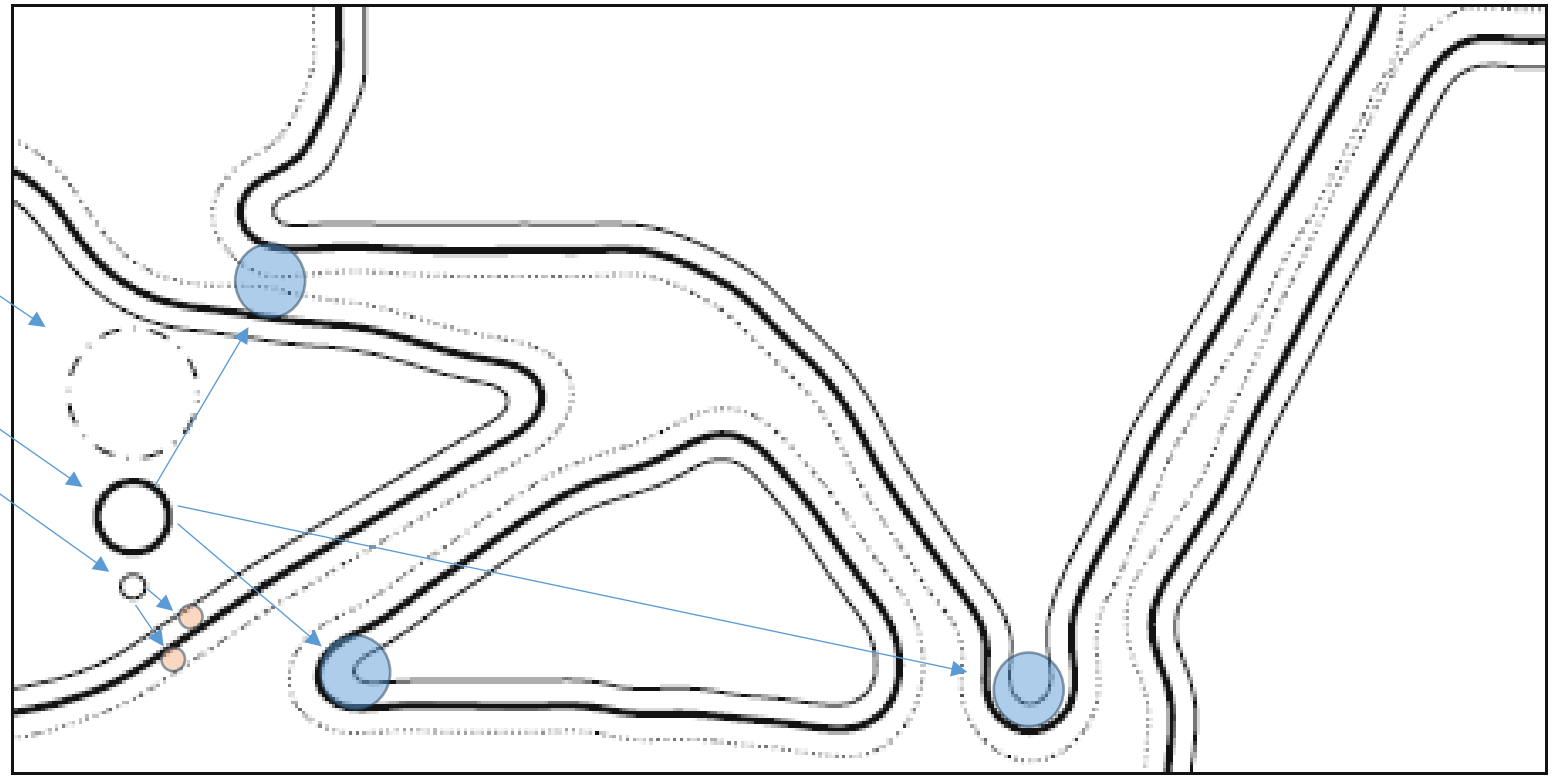
Fig. 12 Normalized length scale on the intermediate design as a function of the threshold η for the robust formulation

Filter size

Minimum length scale in intermediate design

"Manufacturing tolerance"

- Assume the overall topology of the eroded, intermediate and dilated designs are the same.
- If there is a single solid point in the eroded design, there will be a larger area of material in the intermediate design.
- If there is a void point in the dilated design, there will be a void of larger area in the intermediate design.
- (R,η) determines the minimum solid size and void size in the intermediate design.
- η is used as $\eta_d = \eta, \eta_i = 0.5, \eta_e = 1 - \eta$ and determines the difference between the 3 designs and the "manufacturing tolerance".



(a)

Fig. 13 Contours of different designs with $R = 8.4L/200$ and $\eta = 0.2$, The *thin solid lines* denote the contour of the dilated design, the *bold solid lines* denote the contour of the intermediate design, and the *dotted lines* denote the contour of the eroded design. The *dot-dashed circle* indicates the filter support domain, the *bold circle* indicates the minimal length scale on the intermediate design and the *small thin circle* indicates the minimum manufacturing tolerance between the eroded and intermediate designs or the intermediate and the dilated designs. a Compliant inverter.

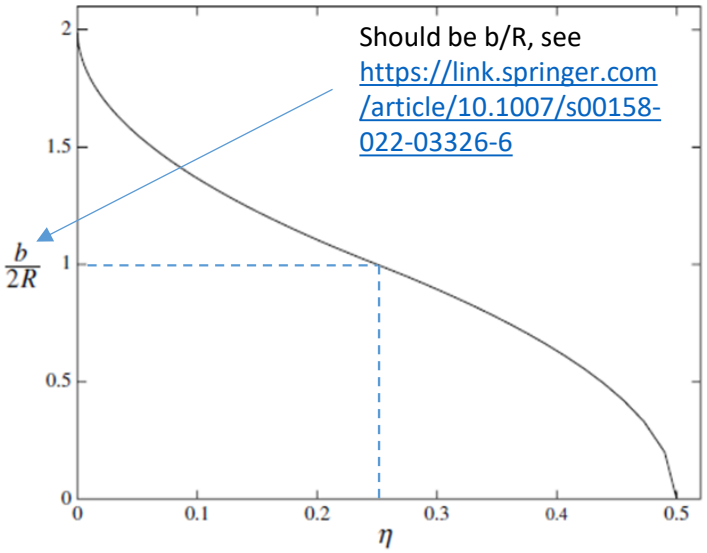


Fig. 12 Normalized length scale on the intermediate design as a function of the threshold η for the robust formulation

The length scale depends on both R and η, so different combination of (R, η) can lead to the same minimum length scale, b.

Even with the same length scale, the two combinations lead to different designs.

Same minimum length scale, *b:*

$$\frac{b}{R} = 2 \Rightarrow b = 2R = 5.6$$

$$\frac{b}{R} = 1 \Rightarrow b = R = 5.6$$

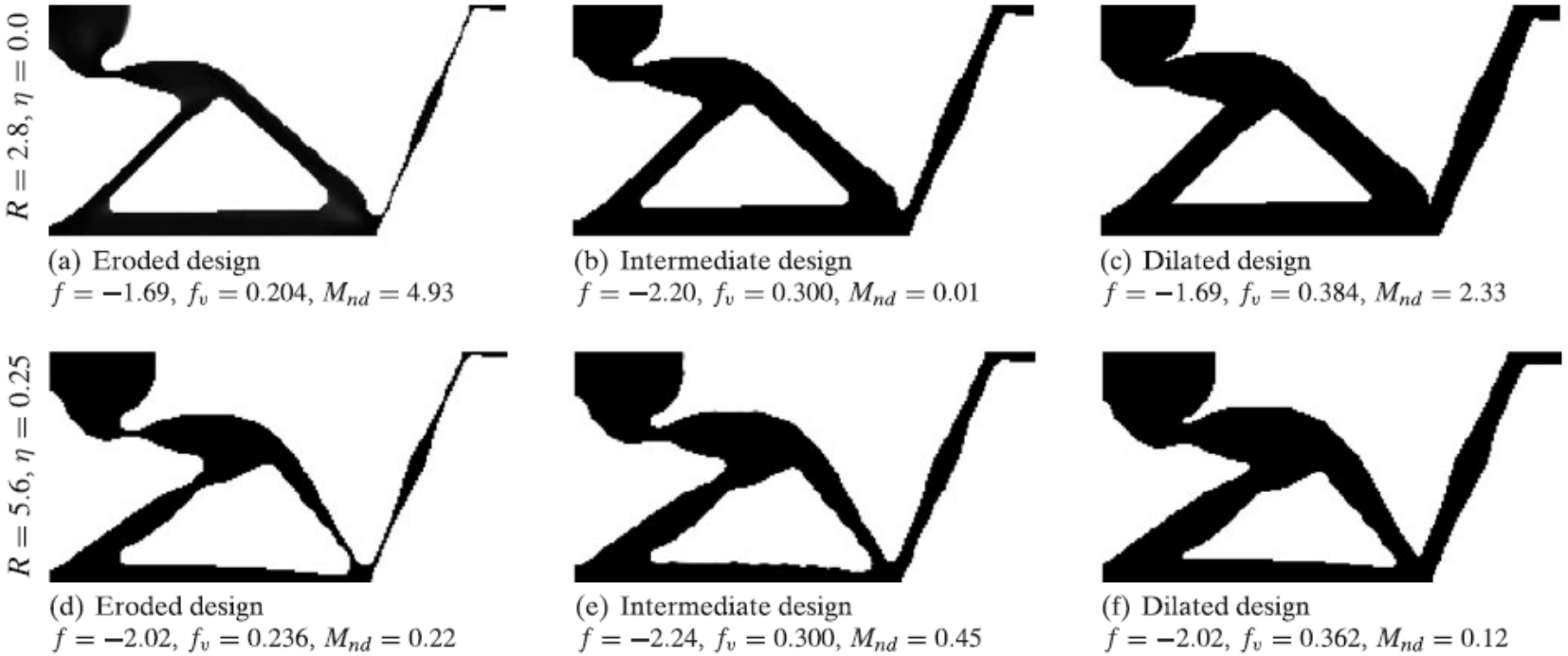


Fig. 14 Robust design of compliant inverter with different filter radius and same minimum length scale

The length scale depends on both R and η, so different combination of (R, η) can lead to the same minimum length scale.
Even with the same length scale, the two combinations lead to different designs.

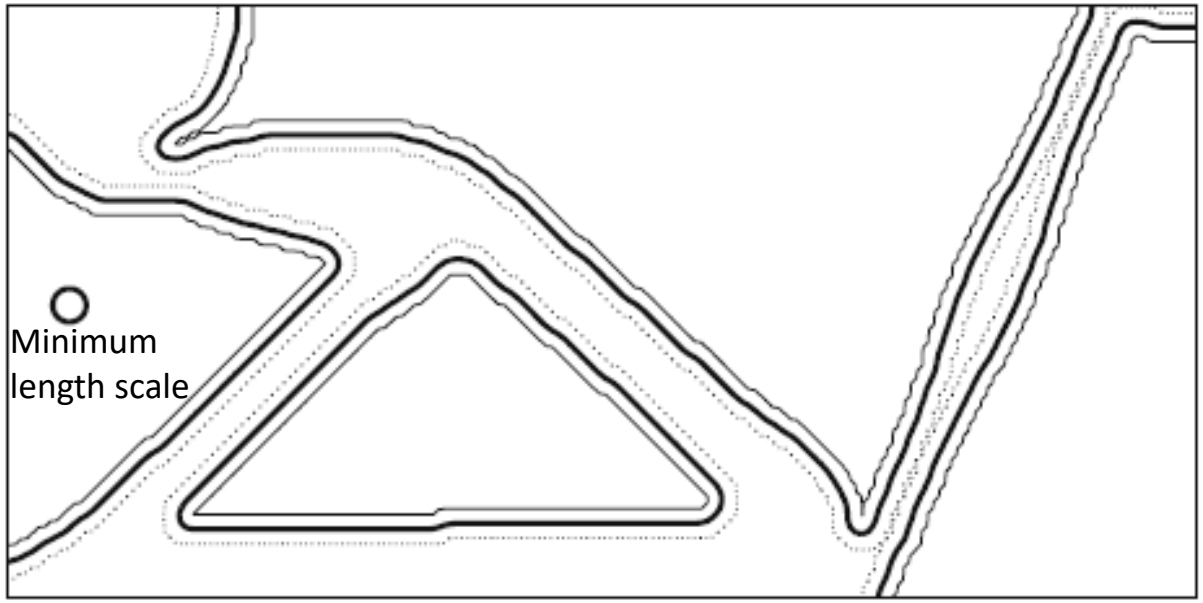
Looking at the contours of the eroded, intermediate and dilated designs, one sees that the difference between them are higher for the small η.

Since a min-max (worst case) problem is solved, all three designs will have good performance.

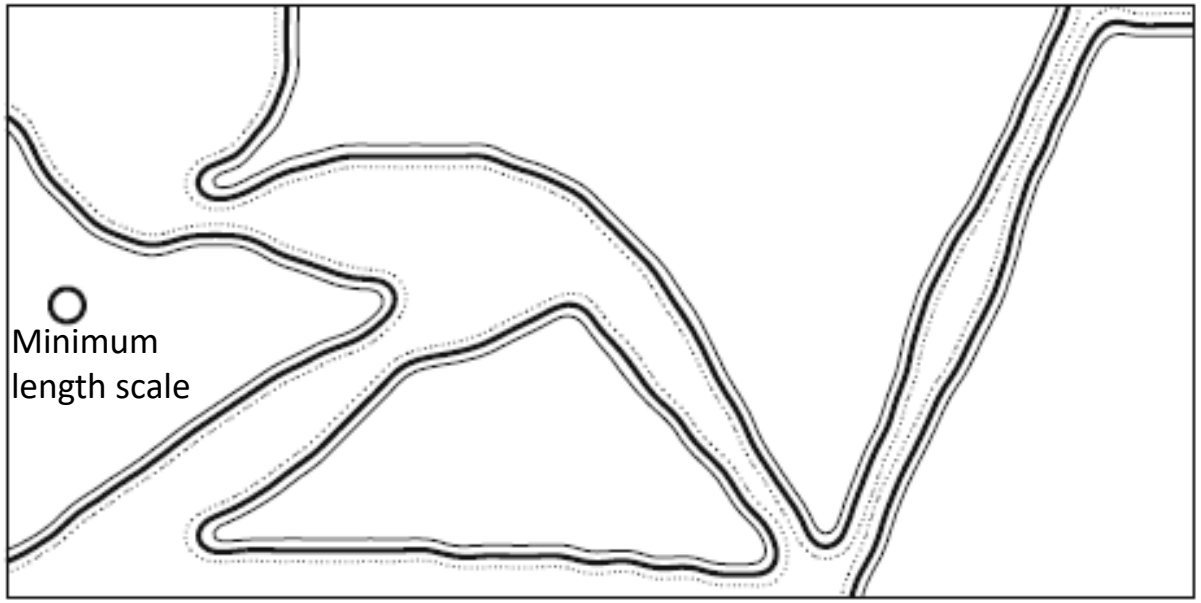
The low η-design is said to be more robust, since the optimization process has considered a larger mount of over- and under-etching as

$$\eta_d = \eta, \eta_i = 0.5\,, \eta_e = 1 - \eta$$

The manufacturing tolerances are thus more forgiving for low η values. The price is a (slightly) worse performing of the η_i "blue-print" design.



(a) $R = 2.8, \eta = 0.0$ f_i=-2.20



(b) $R = 5.6, \eta = 0.25$ f_i=-2.24

Fig. 15 Error tolerance and minimum length scale for the robust compliant inverter design

Helmholtz filter

- Robust topology optimization work also with the Helmholtz filter.
- The Helmholtz radius r in $-r^2\nabla^2\tilde{\rho} + \tilde{\rho} = \rho$ is related to the convolution filter radius R as $r = R/(2\sqrt{3})$.
- The Helmholtz filter is preferred for large filter radius and 3D problems due to memory usage and computation time.
  - It is also preferred in FEniCS, as it is easy to implement a solver for the Helmholtz PDE ☺

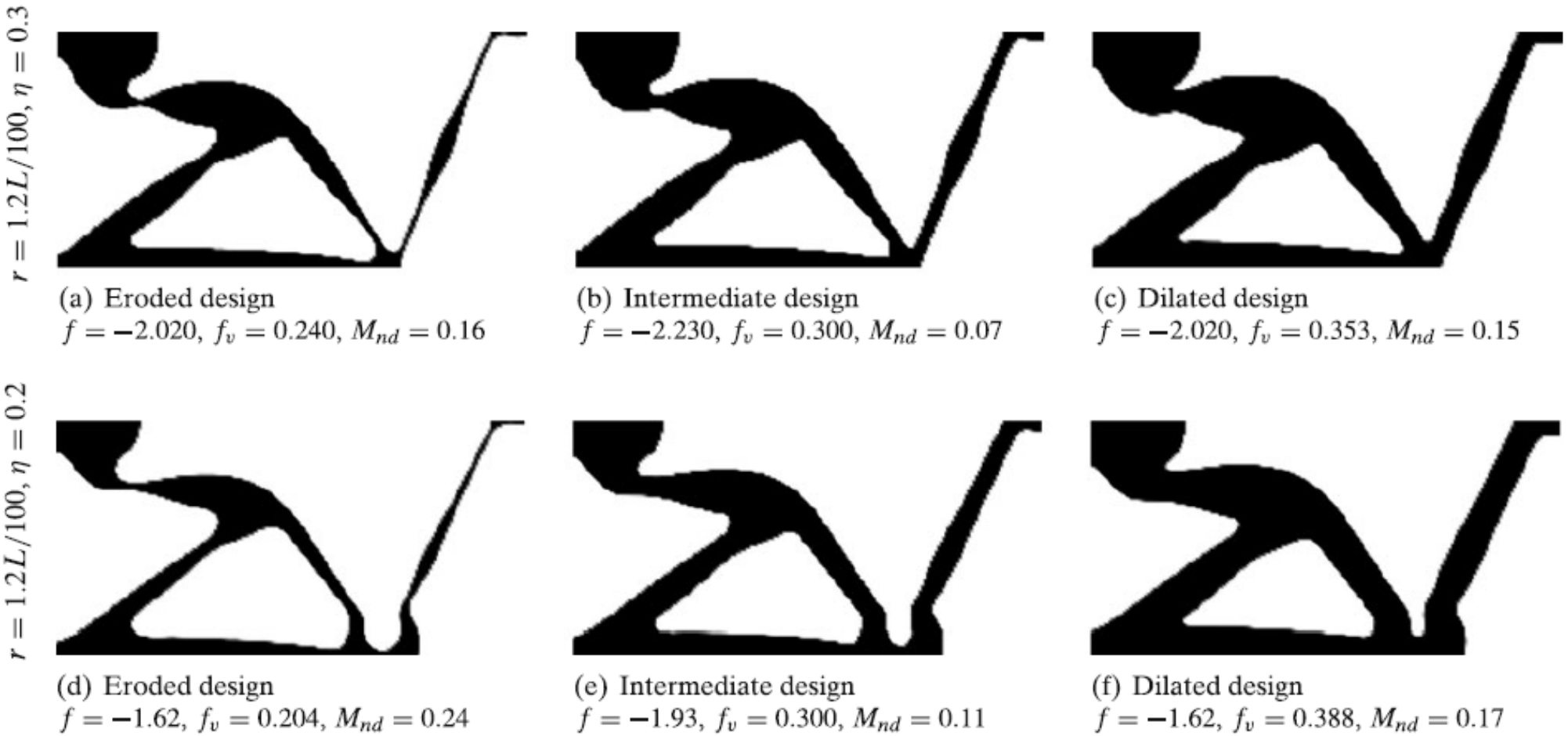


Fig. 17 Robust design of compliant inverter where the filter (4) has been substituted with the Helmholtz filters (15) and (16)

Helmholtz filter

- Refining the mesh creates a higher fidelity solution, with the same overall geometry.

Fig. 18 Robust design (intermediate) obtained with two different mesh sizes with the Helmholtz filter

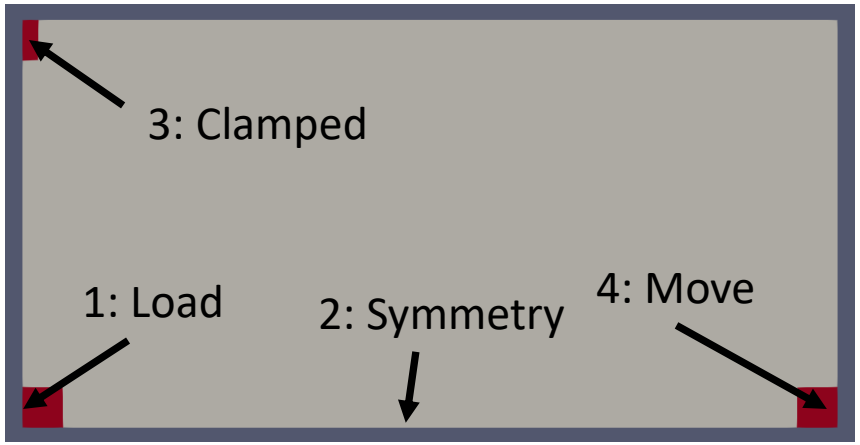
(a) Coarse mesh 20000 elements
$f = -1.93,\ f_v = 0.300,\ M_{nd} = 0.11$

(b) Fine mesh 125000 elements
$f = -1.89,\ f_v = 0.300,\ M_{nd} = 0.07$

Force inverter implementation: Overview

- Copy the cantilever_beam.py file to a new file, e.g., force_inverter.py.
  - Modify the boundary conditions to match the force inverter case.
  - Add a boundary marking for the moving boundary.
  - Add springs to the load and the moving boundary in the solve function.
  - Test the force_inverter.py implementation by, e.g., see that it deforms as expected.
- Copy the Ex7_topopt_elasticity_bridge.py to a new file, e.g., toptop_elasticity_force_inverter.py.
  - Modify the file to import the force_inverter class instead of CantileverBeam class.
  - Remove the redefine_boundary_conditions function.
  - Modify the fixParts function.
  - Modify setUpFunctionals function to change the cost function.
- Make it all work and design awesome force inverters ☺

Force inverter implementation: Boundary conditions (in force_inverter.py)




```
# Define new boundary condition
self.boundary_parts = MeshFunction("size_t", self.mesh, 1)
self.boundary_parts.set_all(0)

self.tol=1e-8
self.LoadSize=W/5
self.ClampSize=W/10
self.MoveSize=W/5

class YSymmetry_boundary(SubDomain):
    def inside(self,x,on_boundary):
        return on_boundary and near(x[1],0)
class generic_boundary_marker(SubDomain):
    def __init__(self,f):
        self.f=f
        SubDomain.__init__(self)
    def inside(self,x,on_boundary):
        return on_boundary and self.f(x)

load_boundary=generic_boundary_marker(self.insideLoad)
load_boundary.mark(self.boundary_parts,1)
clamped_boundary=generic_boundary_marker(self.insideClamped)
clamped_boundary.mark(self.boundary_parts,3)
move_boundary=generic_boundary_marker(self.insideMove)
move_boundary.mark(self.boundary_parts,4)
ysym_boundary=YSymmetry_boundary()
ysym_boundary.mark(self.boundary_parts,2) # Must be marked last to overwrite other markings...

self.traction = F/(W*W)   # Traction, [N/m^2]
self.f = Constant((0, -density*g))             # Body force
self.T = Constant((self.traction,0))
self.spring_kin = self.traction/self.LoadSize  # Spring on input  (load boundary)
self.spring_kout = 0.25*self.spring_kin        # Spring on output (move boundary)

self.ds=Measure('ds',domain=self.mesh,subdomain_data=self.boundary_parts)
self.bc=[DirichletBC(self.V.sub(1), 0, self.boundary_parts,2),DirichletBC(self.V, (0,0), self.boundary_parts,3)]
```

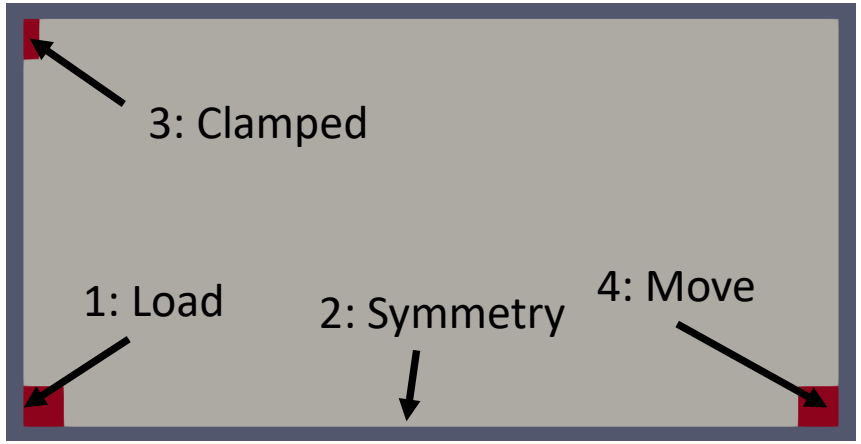
```
def insideClamped(self,x):
    return x[0]<self.ClampSize/2 and x[1]>(self.W-self.ClampSize-self.tol)
def insideLoad(self,x):
    return x[0]<self.LoadSize/2 and x[1]<(self.LoadSize/2+self.tol)
def insideMove(self,x):
    return x[0]>(self.L-self.MoveSize/2-self.tol) and x[1]<(self.MoveSize/2+self.tol)
```

Force inverter implementation: Fix parts (in toptop_elasticity_force_inverter.py)

```
def fixParts(self):
   def InsideFullRegion(x):
       # Return true if coordinates in x is inside a region with rho=1
       return self.mech.insideLoad(x) or self.mech.insideMove(x) or self.mech.insideClamped(x)

   # Get rho's DOF coordinates
   coor = self.FP.Vd.tabulate_dof_coordinates().reshape((-1, self.mech.dim))
   size=self.rho.vector().get_local().shape[0]
   i=0
   rho0=np.zeros(size)
   regions=np.zeros(size)
   for c in coor:
       if InsideFullRegion(c):
           self.mma.xmin[i]=1.0
           self.mma.xmax[i]=1.0
           rho0[i]=1.0
       else:
           rho0[i]=0.5
       i=i+1

   self.rho.vector().set_local(rho0)  # Put values into the rho vector
   self.rho.vector().apply('')        # Needed for parallel execution
   File(self.directory+'/InitialDeisgn.pvd') << self.rho
```



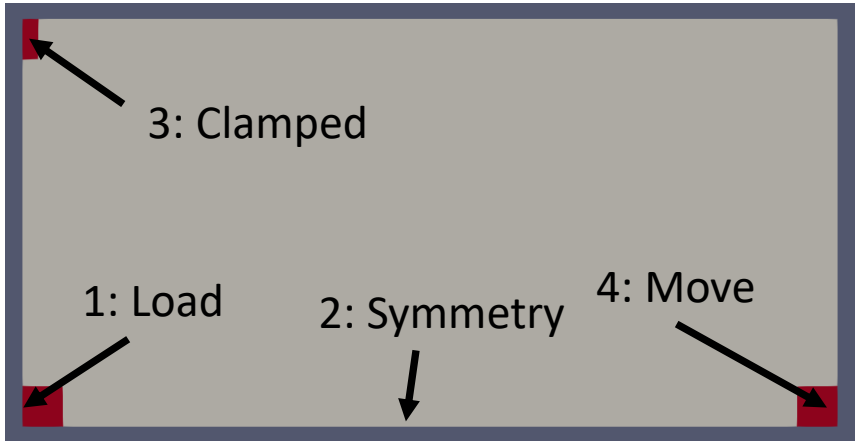
Force inverter implementation: Solver (in force_inverter.py)

```
def solve(self):
    # Define variational problem
    u = TrialFunction(self.V)
    v = TestFunction(self.V)
    a = inner(self.sigma(u), self.epsilon(v))*dx \
        + self.spring_kin*u[0]*v[0]*self.ds(1) \
        + self.spring_kout*u[0]*v[0]*self.ds(4)
    L = self.mask*dot(self.f, v)*dx + dot(self.T, v)*self.ds(1) # body force with mask, traction only on boundary 1

    # Compute solution
    u = Function(self.V,name='Displacement') # 'name' shown in paraview
    if self.useIterativeSolver:
        solve(a == L, u, self.bc, solver_parameters={"linear_solver": self.KrylovSolver, "preconditioner": self.KrylovPrecon})
    else:
        solve(a == L, u, self.bc, solver_parameters={'linear_solver': 'mumps’})
    return u
```

Spring energy: $E(u_x) = \frac{1}{2} k u_x^2$

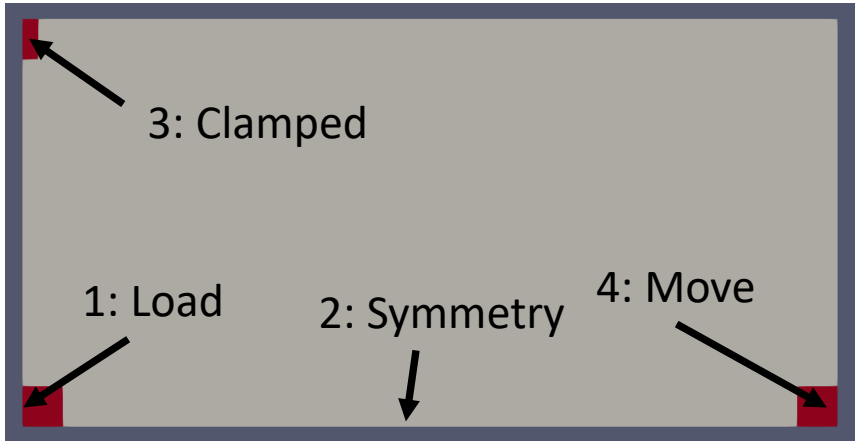
Weak form term: $\left.\frac{dE(u_x+\epsilon v_x)}{d\epsilon}\right|_{\epsilon=0} = \left.\frac{d\left(\frac{1}{2}k(u_x+\epsilon v_x)^2\right)}{d\epsilon}\right|_{\epsilon=0} = k u_x v_x$



Force inverter implementation: Cost and constraint functions

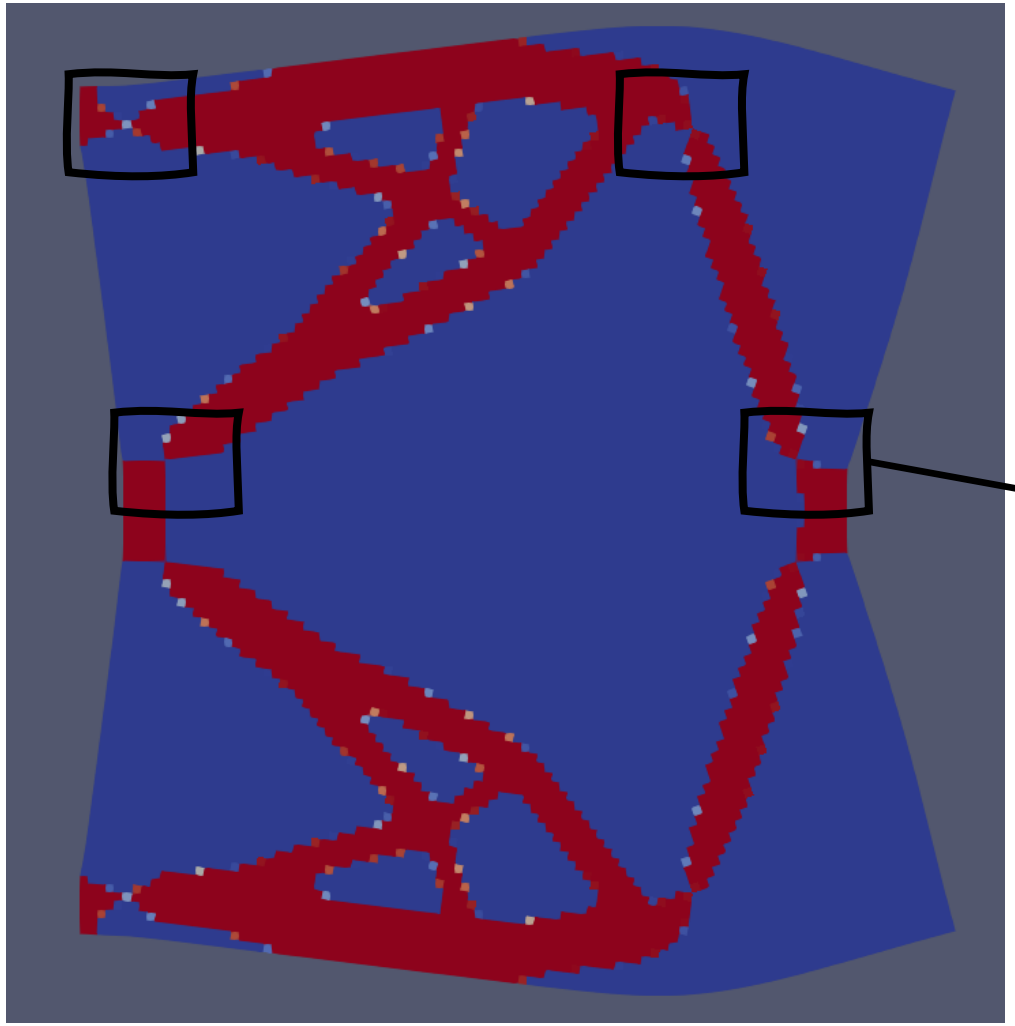
(in toptop_elasticity_force_inverter.py)

```
def setUpFunctionals(self,beta,eta):
     from pyadjoint.tape import annotate_tape
     while not annotate_tape():
         continue_annotation() # Run continue until we actually start annotating...
     tape=get_working_tape()
     tape.clear_tape()
     # Run forward model to record on tape
     (u,rho_bar,rho_tilde)=self.Forward(beta,eta)
     # Make functionals
     m=Control(self.rho)
     J=assemble( u[0]/self.mech.MoveSize*self.mech.ds(4) )
     V=assemble( (rho_bar-self.Volfrac)/self.VolMax*dx )
     # Copy tape, to be able to run optimize such that e.g. mechanics is not solved for when calculating volume.
     tapeJ=tape.copy()
     tapeV=tape.copy()
     self.Jhat = ReducedFunctional(J,m,tape=tapeJ)
     self.Vhat = ReducedFunctional(V,m,tape=tapeV)
     tapeJ.optimize(controls=[m],functionals=[J])
     tapeV.optimize(controls=[m],functionals=[V])
     while annotate_tape():
         pause_annotation() # Run pause until we actually pause annotating...
```

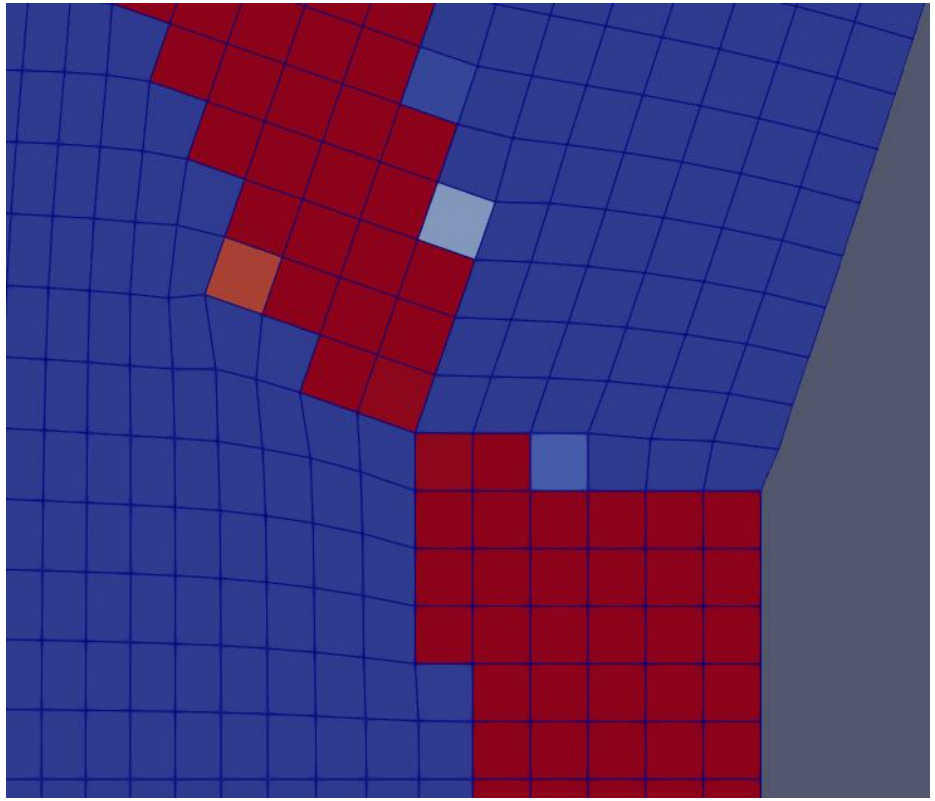


Result

Full geometry (plotted using symmetry) in deformed state.



Several unproducible one-node hinges in solution ☹



Simple robust implementation

- Switch MMA to a min-max problem with 3 ‘cost functions’ and 1 constraint (by calling MMA’s setMinMax(3,1)).
- Run the forward model with 3 different η values.
- Define 3 different functions to measure the motion of each of the 3 designs.
- Define 3 different ‘cost functions’, remembering that all must always give >=0 values for min-max to work. If needed, you can
  - Add a “large enough” constant to all 3 functions.
  - Use expressions such as $(f_0 - f)^2$ which are always positive, with f_0 being a constant and f being the function which measures performance. The minimum is when $f = f_0$.
  - The latter is used in the following.

Simple robust implementation

```
def setUpFunctionals(self,beta):
     from pyadjoint.tape import annotate_tape
     while not annotate_tape():
         continue_annotation() # Run continue until we actually start annotating...
     tape=get_working_tape()
     tape.clear_tape()
     # Run forward model to record on tape
     (u_N,rho_bar_N,rho_tilde_N)=self.Forward(beta,self.eta_N)
     (u_D,rho_bar_D,rho_tilde_D)=self.Forward(beta,self.eta_D)
     (u_E,rho_bar_E,rho_tilde_E)=self.Forward(beta,self.eta_E)
     # Make functionals
     m=Control(self.rho)
     J_N=assemble( u_N[0]/self.mech.MoveSize*self.mech.ds(4) )
     J_D=assemble( u_D[0]/self.mech.MoveSize*self.mech.ds(4) )
     J_E=assemble( u_E[0]/self.mech.MoveSize*self.mech.ds(4) )
     V_D=assemble( (rho_bar_D-self.Volfrac)/self.VolMax*dx )
     # Copy tape, to be able to run optimize such that e.g. mechanics
     # is not solved for when calculating volume.
     tapeJ=tape.copy()
     tapeV=tape.copy()
     self.J_Nhat = ReducedFunctional(J_N,m,tape=tapeJ)
     self.J_Dhat = ReducedFunctional(J_D,m,tape=tapeJ)
     self.J_Ehat = ReducedFunctional(J_E,m,tape=tapeJ)
     self.V_Dhat = ReducedFunctional(V_D,m,tape=tapeV)
     tapeJ.optimize(controls=[m],functionals=[J_N,J_D,J_E])
     tapeV.optimize(controls=[m],functionals=[V_D])
     while annotate_tape():
         pause_annotation() # Run pause until we actually pause annotating...
```

Notation used in paper is:

$$\eta_d = \eta, \qquad \eta_i = 0.5\,, \qquad \eta_e = 1 - \eta$$

Notation used here:

$$\eta_D = \eta_N - \Delta\eta, \qquad \eta_N = 0.5\,, \qquad \eta_E = \eta_N + \Delta\eta$$

```
class topOpt():
   def __init__(self):
     …Some parts omitted…
        self.delta_eta=0.2
        self.eta_N=0.5
        self.eta_D=self.eta-self.delta_eta
        self.eta_E=self.eta+self.delta_eta
     …Some parts omitted…
```

Volfrac is the volume fraction for the dilated design.
It is not updated every 20 iteration as in the paper.

Simple robust implementation

```
def f(self,x):
    """Redefine function from MMA that calculates the vector function f"""
    self.rho.vector().set_local(x.getArray()[:])
    self.rho.vector().apply('')

    J_N=self.J_Nhat(self.rho)
    J_D=self.J_Dhat(self.rho)
    J_E=self.J_Ehat(self.rho)
    self.Js=[J_N,J_D,J_E] # Save Js for use in g
    self.move_max=-0.1    # Amount we want it to move
    V_D=self.V_Dhat(self.rho)
    return np.array([0,
                     (self.move_max-J_N)**2,
                     (self.move_max-J_D)**2,
                     (self.move_max-J_E)**2,
                     V_D])

 def g(self,x):
    """Redefine function from MMA that calculates the gradient (Jacobian) of f"""
    self.rho.vector().set_local(x.getArray()[:])
    self.rho.vector().apply('')
    J_N,J_D,J_E=self.Js[:] # g is run right after f inside MMA, so no need to calculate the Js again!
    dc_N=self.J_Nhat.derivative().vector().get_local()
    dc_D=self.J_Dhat.derivative().vector().get_local()
    dc_E=self.J_Ehat.derivative().vector().get_local()
    dv_D=self.V_Dhat.derivative().vector().get_local()
    return np.array([0*dc_N,
                     -2*(self.move_max-J_N)*dc_N,
                     -2*(self.move_max-J_D)*dc_D,
                     -2*(self.move_max-J_E)*dc_E,
                     dv_D])
```

Dolfi-adjoint can only differentiate functions based on integrals over the domain and/or boundary:

$$f = \int_V F_V dx + \int_S F_s ds$$

To calculate the gradient of, say,

$$g = (f_0 - f)^2$$

we can use the chain rule ourselves:

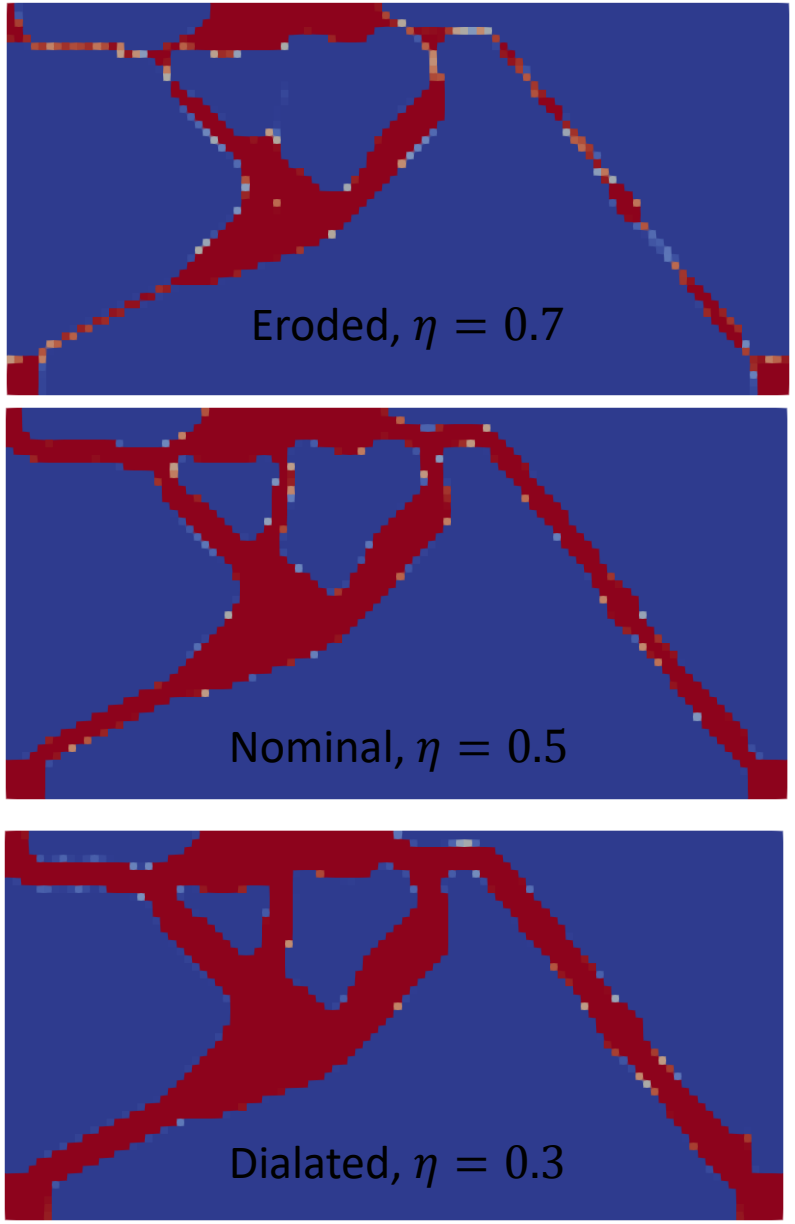
$$\nabla g = -2(f_0 - f)\nabla f$$

∇f can be calculated by dolfin-adjoint.

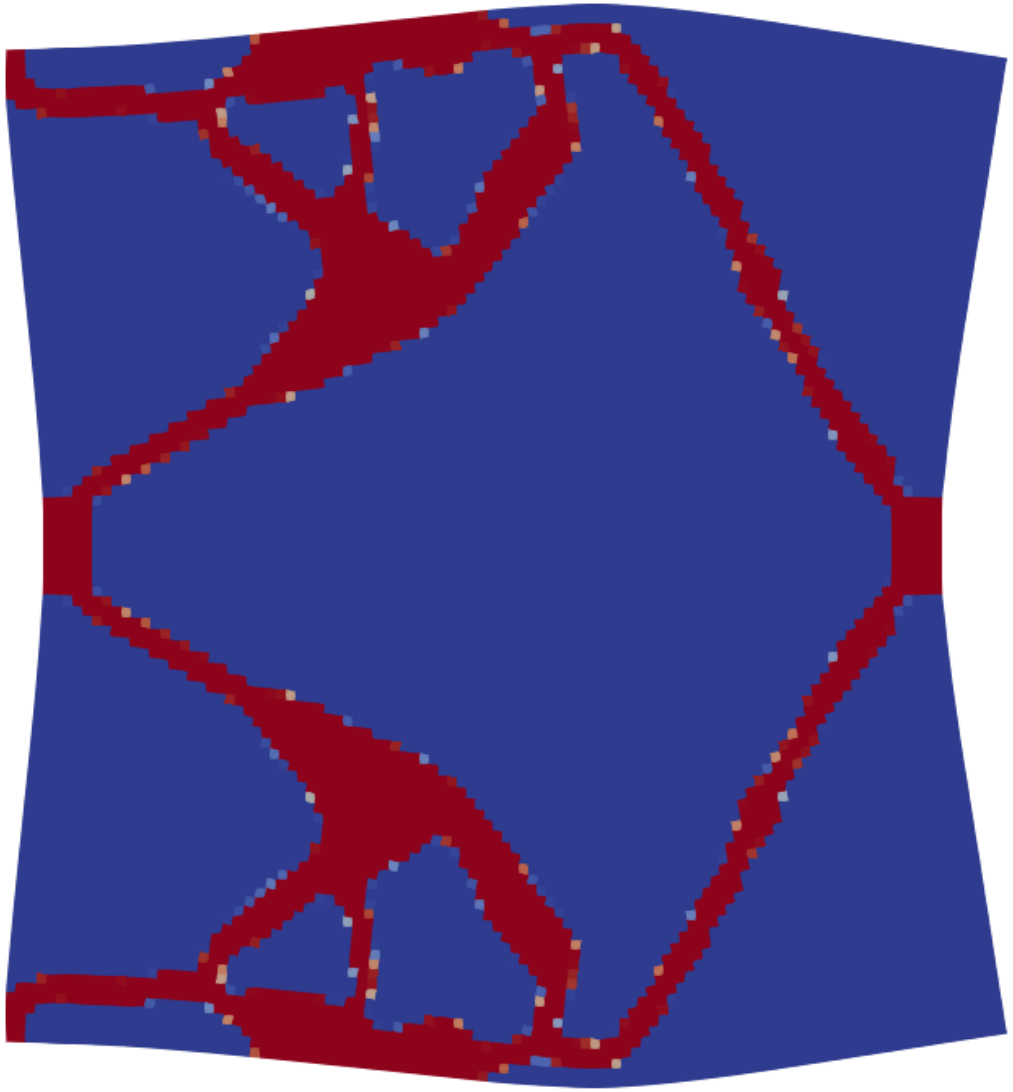
Remember:

```
def setUpOptimizer(self,x0):
    """Initializes MMA"""
    …
    # MMA setup
    # Switch MMA to min-max optimization problem.
    self.mma.setMinMax(3,1)
    …
```

Result of robust optimization



Full Nominal (intermediate) design warped by displacement field.

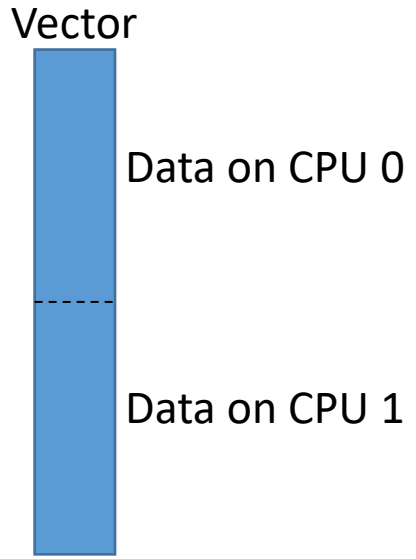


No hinges ☺
Solution time tripled ☹

Parallelization

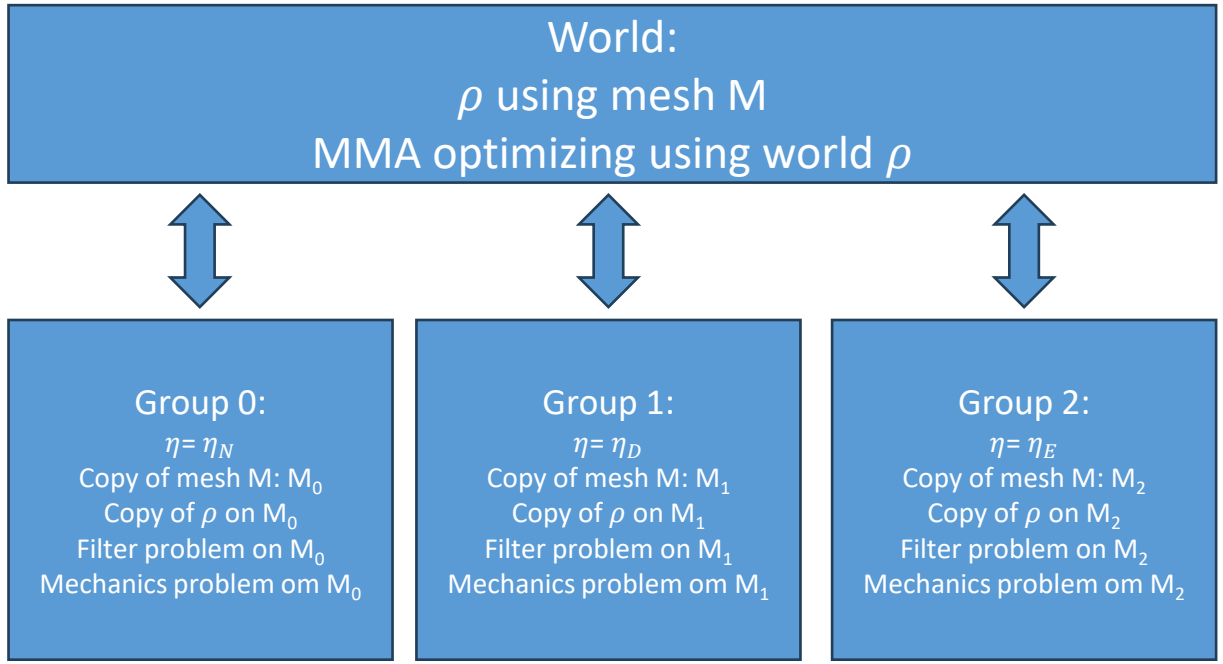
- Robust topology optimization solved the issue with the hinges, but it takes ~3x the time to solve the robust optimization problem ☹
- Modern computer often have 4 or more physical (performance) cores. We can solve one problem on a different core to solve all 3 problems in parallel, and the solution time should be roughly the same as the non-robust optimization ☺
- MPI works with communicators which is a group of processes that can talk together.
- All processes run the same program but may work on different parts of the same vector or matrix or other data.
- We have until now used
  - MPI.comm_world – all N processes specified with '-n N' to 'mpiexec' or 'mpirun'.
  - Defining a vector using 'world' splits it into N parts, one for each process.
- Another common communicator is
  - MPI.comm_self – only the process itself.
  - Defining a vector using 'self' creates one vector on each process.
- A simple way to parallelize a program is to use MPI.comm_world for all vector and matrices.
  - This is the default in FEniCS.
  - Some problems run on N processes will be solved N times faster.
  - Many problems don't show an N times speedup when run on N cores, especially small problems.
  - Communication and other overhead related to parallelization prevent the ideal speedup.

To execute a program in parallel using 2 cores, write e.g.
mpiexec -n 2 python3 topopt_program.py



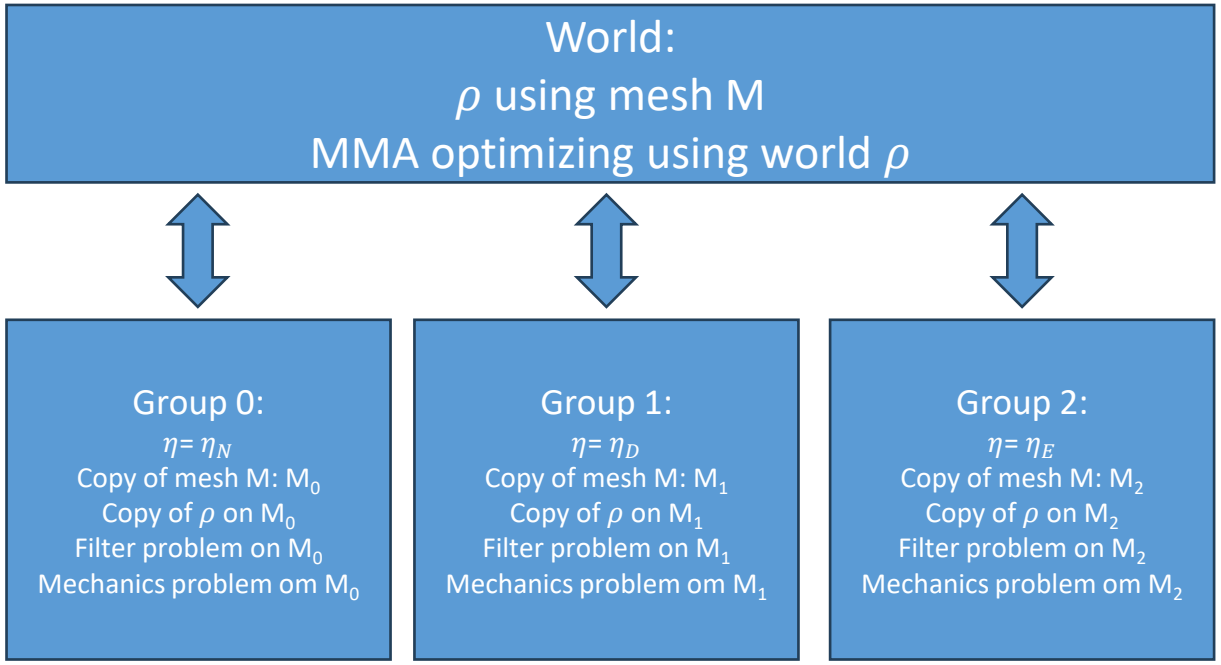
Parallelization

- Communicators can be split into groups.
  - If you run on N CPU cores and split into 3 groups, each group will be run on N/3 cores.
  - You need at least 3 cores to split into 3 groups!
- Each group can then handle one instance of a problem.
- The robust optimization had 3 instances of the geometry, and 3 groups are thus created.
- All groups have access to the same 'bare' design variables, ρ, defined on world.
- Each group have a unique η-value.
- Each group also have a "group copy" of the mesh, the design variables ρ, the filtering problem and the mechanics problem.
- Each group will solve the filtering and mechanics problem, corresponding to the group's η-value, and calculate the group cost and constraint function values and gradients.
- MMA works with the ρ defined on world.
- The results from the groups are communicated to world to be used in MMA to update ρ.

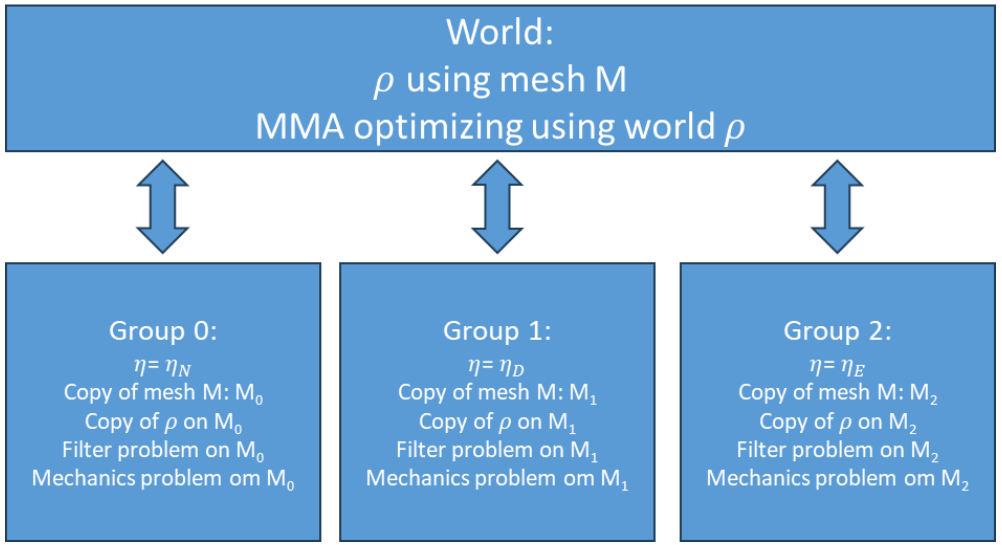


Parallelization

- The mechanics problem must be able to work with a custom communicator.
- The mesh of the 3 instances must be the same.
- To ensure this, the mesh can be created, saved in a file and read in again in each of the 3 instances.
- While the world mesh and group meshes are the same, FEniCS may arrange the DOFs differently.
- A mapping between the Group and World DOFs must be made.



Parallelization



- A 'Parallel' helper class has been defined.
- The class is instanciated as

```
self.parallel=Parallel(MPI.comm_world,3)
```

  - This splits the World into 3 groups, numbered 0, 1, 2.
- The group communicator is found as

```
self.parallel.group_comm
```

- The mapping between two DG0 function spaces can be created as

```
self.parallel.create_mapping(self.Vd,self.FP_group.Vd)
```

- Scalars calculated in a group can be communicated to World using

```
self.Js=self.parallel.sgroup2global(J)
```

  - self.Js contains 3 scalars, as 3 groups were defined.
- Vectors calculated in a group can be communicated to World using

```
dc=self.parallel.vgroup2global(self.Jhat.derivative().vector())
```

  - dc contains 3 vectors and is of size 3 x length(vector).
    Replace 3 with the number of groups you define when creating an instance of Parallel ☺
- FEniCS functions calculated on world can be communicated to a group using

```
self.parallel.fglobal2group(self.rho,self.rho_group)
```

- Importing Parallel and making a few changes to the non-robust code makes a parallel robust version ☺
- Some code from an implementation is shown on the next few slides…

Parallelization - Some parts of an implementation

```
def __init__(self):
    … Some parts skipped …
    self.mech=ForceInverter(L,W,self.E0,self.nu,F,0,mesh_size,comm=MPI.comm_world) # Create the mechanics problem on world to get the mesh.
    # Write mesh for groups
    meshfile=self.directory+"/mesh.xdmf"
    with XDMFFile(self.mech.comm,meshfile) as f:
        f.write(self.mech.mesh)

    # Define group instances
    self.parallel=Parallel(MPI.comm_world,3) # Create 3 groups, numbered 0, 1 and 2
    self.mech_group=ForceInverter(L,W,self.E0,self.nu,F,0,mesh_size,comm=self.parallel.group_comm,meshfile=meshfile)

    # Create rho on global communicator for MMA to work with
    self.Vd=FunctionSpace(self.mech.mesh,'DG',0)
    self.rho=interpolate(Constant(self.Volfrac),self.Vd)
    self.rho.rename('rho','rho')
    self.rho_petsc=as_backend_type(self.rho.vector()).vec()  # Pointer to rho's petsc data structure

    # Create rho on group communicator
    self.FP_group=FilterAndProject(R,self.mech_group.mesh) # Using group mesh also means using the group communicator
    self.rho_group=interpolate(Constant(self.Volfrac),self.FP_group.Vd)
    self.rho_group.rename('rho group','rho group')
    self.parallel.create_mapping(self.Vd,self.FP_group.Vd) # Create mapping between world and group DOFs.

    # delta eta for robust optimization
    self.eta=0.5
    self.delta_eta=0.2
    if self.parallel.group==0:
        # Nominal design in group 0
        self.group_eta=self.eta
        self.eta_name='N'
    elif self.parallel.group==1:
        # Dialated design in group 1
        self.group_eta=self.eta-self.delta_eta
        self.eta_name='D'
    elif self.parallel.group==2:
        # Eroded design in group 2
        self.group_eta=self.eta+self.delta_eta
        self.eta_name='E'
```

```
def Forward(self,beta):
    rho_tilde=self.FP_group.filter(self.rho_group)
    rho_bar,_=self.FP_group.Heavi_project(beta,self.group_eta,rho_tilde)
    E=self.Emin+pow(rho_bar,self.pen)*(self.E0-self.Emin) # SIMP E
    self.mech_group.E=E
    self.mech_group.mask=rho_bar
    u=self.mech_group.solve()
    return (u,rho_bar,rho_tilde)
```

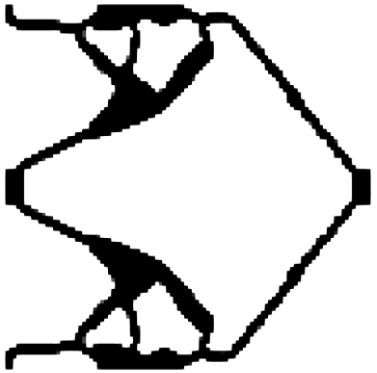
Parallelization - Some parts of an implementation

```
def f(self,x):
     """Redefine function from MMA that calculates the vector function f"""
     self.rho.vector().set_local(x.getArray()[:])
     self.rho.vector().apply('')
     # Transfer rho from world (global) to group
     self.parallel.fglobal2group(self.rho,self.rho_group)
     # Calculate function values on group
     J=self.Jhat(self.rho_group)
     V=self.Vhat(self.rho_group)
     # Transfer scalars from group to world (global)
     self.Js=self.parallel.sgroup2global(J)
     self.Vs=self.parallel.sgroup2global(V)
     J_N,J_D,J_E=self.Js[:]
     V_N,V_D,V_E=self.Vs[:]
     self.move_max=-0.1    # Amount we want it to move
     return np.array([0,
                      (self.move_max-J_N)**2,
                      (self.move_max-J_D)**2,
                      (self.move_max-J_E)**2,
                      V_D])

 def g(self,x):
     """Redefine function from MMA that calculates the gradient (Jacobian) of f"""
     self.rho.vector().set_local(x.getArray()[:])
     self.rho.vector().apply('')
     # Transfer rho from world (global) to group
     self.parallel.fglobal2group(self.rho,self.rho_group)
     # Calculate gradients and transfer vectors from group to world (global)
     dc=self.parallel.vgroup2global(self.Jhat.derivative().vector())
     dv=self.parallel.vgroup2global(self.Vhat.derivative().vector())
     J_N,J_D,J_E=self.Js[:]
     V_N,V_D,V_E=self.Vs[:]
     dc_N,dc_D,dc_E=dc[0:3,:]
     dv_N,dv_D,dv_E=dv[0:3,:]
     return np.array([0*dc_N,
                      -2*(self.move_max-J_N)*dc_N,
                      -2*(self.move_max-J_D)*dc_D,
                      -2*(self.move_max-J_E)*dc_E,
                      dv_D])
```

Results of optimization with a parallel robust implementation should be very similar to simple robust implementation, but running time should ideally be 1/3. Not quite that good in practice!

Simple parallelization using 3 CPUs: 34 minutes



Using parallel.py with 3 CPUs: 14 minutes

Conclusions

- "Standard" filtering and projection can only guarantee a feature size of either solid (using $\eta = 0$) or void (using $\eta = 1$), for a filter with compact support.
- Robust topology optimization will set a feature size of both the solid and void phases by using 3 different projection levels, η_D, η_N and η_E.
- The projection levels can be defined as

$$\eta_d = \eta, \qquad \eta_i = 0.5\,, \qquad \eta_e = 1 - \eta$$

  or alternatively as

$$\eta_D = \eta_N - \Delta\eta, \qquad \eta_N = 0.5\,, \qquad \eta_E = \eta_N + \Delta\eta$$

- The robust approach can remove very thin or mesh-element-sized hinges in compliant mechanism design, by simultaneously constraining both the void and solid phases.
- The robust approach optimize for the worst case of the 3 design realizations.
- A combination of the filter radius and η control the minimum feature size and the manufacturing tolerance ("difference between the eroded and dilated designs").
- The robust approach also works well for the Helmholtz filter.
- The penalty to pay is that 3 FEM problems must be solved to evaluate, e.g., the cost function.
- Advanced implementations will run the 3 FEM problems in parallel, such that no extra real-time is used, only extra CPU-time and computer memory.
- We will see later that this robust approach can also fail for some problems…
  - A solution is to define the dilated, nominal and eroded geometries slightly differently.